

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Exp No: 1

Date: 10-07-2026

Name: Kushaal Shyam Potta

Reg No.: 3122235001071

Implementation of lexical analyser and symbol table

Aim:

To build a lexical analyzer (scanner) in C that reads a C++ source program, recognizes and classifies its tokens (keywords, identifiers, constants, operators, special characters, comments and function calls), and records the details of each identifier in a symbol table.

Tools Used:

- C language
- GCC

Code:

scanner.c

```

/* -----
Lexical Analyzer + Symbol Table Builder (for C++ programs)
Reads source.cpp, classifies tokens, builds symbol table.
----- */

#include <stdio.h>

#include <string.h>

#include <ctype.h>

#define MAX_SRC 6000

#define MAX_TOK 64

#define MAX_SYMS 100

static char src[MAX_SRC];

```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
static int srcLen = 0;
```

```
static int pos = 0;
```

```
typedef struct {
```

```
    char name[32];
```

```
    char type[12];
```

```
    int bytes;
```

```
    int address;
```

```
    char value[32];
```

```
} SymEntry;
```

```
static SymEntry table[MAX_SYMS];
```

```
static int symCount = 0;
```

```
static int addrCounter = 1000;
```

```
static const char *reserved[] = {
```

```
    "auto", "bool", "break", "case", "catch", "char", "class", "const", "constexpr",
```

```
    "continue", "default", "delete", "do", "double", "else", "enum", "explicit",
```

```
    "extern", "false", "float", "for", "friend", "goto", "if", "inline", "int", "long",
```

```
    "mutable", "namespace", "new", "nullptr", "operator", "private", "protected",
```

```
    "public", "register", "return", "short", "signed", "sizeof", "static", "struct",
```

```
    "switch", "template", "this", "throw", "true", "try", "typedef", "typename",
```

```
    "union", "unsigned", "using", "virtual", "void", "volatile", "while"
```

```
};
```

```
static const int reservedCount = sizeof(reserved) / sizeof(reserved[0]);
```

```
static int sizeOfType(const char *t) {
```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (!strcmp(t, "char")) return 1;
    if (!strcmp(t, "bool")) return 1;
    if (!strcmp(t, "float")) return 4;
    if (!strcmp(t, "double")) return 8;
    return 2;
}

static int isReserved(const char *word) {
    for (int i = 0; i < reservedCount; i++)
        if (!strcmp(word, reserved[i])) return 1;
    return 0;
}

static void recordSymbol(const char *name, const char *type) {
    if (symCount >= MAX_SYMS) return;
    strcpy(table[symCount].name, name);
    strcpy(table[symCount].type, type);
    table[symCount].bytes = sizeofType(type);
    table[symCount].address = addrCounter;
    addrCounter += table[symCount].bytes;
    strcpy(table[symCount].value, "-");
    symCount++;
}

static int peek(int offset) {
    int i = pos + offset;
    return (i < srcLen) ? src[i] : '\0';
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

}

```
static void skipBlanksFrom(int *idx) {
    while (*idx < srcLen && isspace((unsigned char)src[*idx])) (*idx)++;
}
```

```
static void handlePreprocessor(void) {
    printf("#");
    pos++;
    while (pos < srcLen && src[pos] != '\n') putchar(src[pos++]);
    printf(" - preprocessor directive\n");
}
```

```
static int handleComment(void) {
    if (peek(0) == '/' && peek(1) == '/') {
        while (pos < srcLen && src[pos] != '\n') pos++;
        return 1;
    }
    if (peek(0) == '/' && peek(1) == '*') {
        pos += 2;
        while (pos < srcLen && !(src[pos] == '*' && peek(1) == '/')) pos++;
        if (pos < srcLen) pos += 2;
        else printf("Error: unterminated comment\n");
        return 1;
    }
    return 0;
}
```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
static char activeType[12] = "";
```

```
static void handleWordToken(void) {
```

```
    char word[MAX_TOK];
```

```
    int i = 0;
```

```
    while (isalnum((unsigned char)src[pos]) || src[pos] == '_')
```

```
        word[i++] = src[pos++];
```

```
    word[i] = '\0';
```

```
    int look = pos;
```

```
    skipBlanksFrom(&look);
```

```
    if (isReserved(word)) {
```

```
        printf("%s - keyword\n", word);
```

```
        if (!strcmp(word,"int") || !strcmp(word,"char") || !strcmp(word,"float") ||
```

```
            !strcmp(word,"double") || !strcmp(word,"long") || !strcmp(word,"short") ||
```

```
            !strcmp(word,"bool"))
```

```
            strcpy(activeType, word);
```

```
        else
```

```
            activeType[0] = '\0';
```

```
        return;
```

```
    }
```

```
    if (src[look] == '(') {
```

```
        printf("%s", word);
```

```
        pos = look;
```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

while (pos < srcLen && src[pos] != ')') putchar(src[pos++]);
if (pos < srcLen) { putchar(src[pos]); pos++; }
printf(" - function call\n");
activeType[0] = '\0';
return;
}

```

```

printf("%s - identifier\n", word);
if (activeType[0] != '\0') {
    recordSymbol(word, activeType);
    int r = pos;
    skipBlanksFrom(&r);
    if (src[r] == '=') {
        printf("= - assignment operator\n");
        r++;
        skipBlanksFrom(&r);
        char val[MAX_TOK]; int j = 0;
        while (r < srcLen && src[r] != ';' && src[r] != '\n') val[j++] = src[r++];
        val[j] = '\0';
        printf("%s - constant\n", val);
        strcpy(table[symCount - 1].value, val);
        pos = r;
    }
}
}
}

```

```

static void handleNumberToken(void) {

```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
char num[MAX_TOK];
```

```
int i = 0;
```

```
if (src[pos] == '0' && (peek(1) == 'x' || peek(1) == 'X')) {
```

```
    num[i++] = src[pos++];
```

```
    num[i++] = src[pos++];
```

```
    while (isxdigit((unsigned char)src[pos])) num[i++] = src[pos++];
```

```
    num[i] = '\0';
```

```
    printf("%s - hexadecimal constant\n", num);
```

```
    return;
```

```
}
```

```
while (isdigit((unsigned char)src[pos])) num[i++] = src[pos++];
```

```
if (src[pos] == '.') {
```

```
    num[i++] = src[pos++];
```

```
    while (isdigit((unsigned char)src[pos])) num[i++] = src[pos++];
```

```
    num[i] = '\0';
```

```
    printf("%s - double constant\n", num);
```

```
    return;
```

```
}
```

```
num[i] = '\0';
```

```
printf("%s - integer constant\n", num);
```

```
}
```

```
static void handleStringToken(void) {
```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

char str[100];

int i = 0;

str[i++] = src[pos++];

while (pos < srcLen && src[pos] != "" && src[pos] != '\n' && i < 98) {

    if (src[pos] == '\\' && pos + 1 < srcLen) str[i++] = src[pos++]; /* escape */

    str[i++] = src[pos++];

}

if (src[pos] == "") str[i++] = src[pos++];

str[i] = '\0';

printf("%s - string constant\n", str);

}

```

```

static void handleCharToken(void) {

    char ch[8];

    int i = 0;

    ch[i++] = src[pos++];

    while (pos < srcLen && src[pos] != '\"' && src[pos] != '\n' && i < 6) {

        if (src[pos] == '\\' && pos + 1 < srcLen) ch[i++] = src[pos++];

        ch[i++] = src[pos++];

    }

    if (src[pos] == '\"') ch[i++] = src[pos++];

    ch[i] = '\0';

    printf("%s - character constant\n", ch);

}

```

```

static int handleTwoCharOperator(void) {

    char c0 = src[pos], c1 = peek(1);

```


Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

if ((c0=='+'&&c1=='=') || (c0=='-'&&c1=='=') || (c0=='*'&&c1=='=') ||
    (c0=='/'&&c1=='=') || (c0=='%'&&c1=='=')) {
    printf("%c%c - arithmetic assignment operator\n", c0, c1); pos += 2; return 1;
}

if ((c0=='+'&&c1=='+') || (c0=='-'&&c1=='-')) {
    printf("%c%c - unary operator\n", c0, c1); pos += 2; return 1;
}

if (c0=='&'&&c1=='&') { printf("& - logical operator\n"); pos+=2; return 1; }
if (c0=='|'&&c1=='|') { printf("|| - logical operator\n"); pos+=2; return 1; }
if (c0=='='&&c1=='=') { printf("== - relational operator\n"); pos+=2; return 1; }
if (c0=='!'&&c1=='=') { printf("!= - relational operator\n"); pos+=2; return 1; }
if (c0=='<'&&c1=='=') { printf("<= - relational operator\n"); pos+=2; return 1; }
if (c0=='>'&&c1=='=') { printf(">= - relational operator\n"); pos+=2; return 1; }
if (c0=='<'&&c1=='<') { printf("<< - bitwise / stream insertion operator\n"); pos+=2; return 1; }
if (c0=='>'&&c1=='>') { printf(">> - bitwise / stream extraction operator\n"); pos+=2; return 1; }
if (c0==':'&&c1==':') { printf(":: - scope resolution operator\n"); pos+=2; return 1; }
if (c0=='-'&&c1=='>') { printf("-> - member access operator\n"); pos+=2; return 1; }

return 0;
}

```

```

static void handleSingleCharOperator(void) {
    char c = src[pos];

    if (c=='+' || c=='-' || c=='*' || c=='/' || c=='%') { printf("%c - arithmetic operator\n", c); pos++; return; }
    if (c=='<' || c=='>') { printf("%c - relational operator\n", c); pos++; return; }
    if (c=='=') { printf("=" - assignment operator\n"); pos++; return; }
    if (c=='!') { printf("! - logical operator\n"); pos++; return; }
}

```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (c=='?' || c==':')          { printf("%c - conditional operator\n", c); pos++; return; }
    if (c=='&' || c=='|' || c=='^') { printf("%c - bitwise operator\n", c); pos++; return; }
}

```

```

static int handleSpecialChar(void) {
    if (strchr(";,.[>(){}", src[pos]) != NULL) {
        printf("%c - special character\n", src[pos]);
        if (src[pos] == ';') activeType[0] = '\0';
        pos++;
        return 1;
    }
    return 0;
}

```

```

static void printSymbolTable(void) {
    printf("\nContent of Symbol Table\n");
    printf("%-15s %-8s %-12s %-10s %-10s\n", "Identifier", "Type", "No of bytes", "Address", "Value");
    for (int k = 0; k < symCount; k++)
        printf("%-15s %-8s %-12d %-10d %-10s\n",
            table[k].name, table[k].type, table[k].bytes, table[k].address, table[k].value);
}

```

```

static int loadSource(const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (!fp) return 0;
    srcLen = (int)fread(src, 1, MAX_SRC - 1, fp);
    src[srcLen] = '\0';
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

fclose(fp);

return 1;
}

int main(void) {
    if (!loadSource("source.cpp")) {
        printf("cannot open source.cpp\n");
        return 1;
    }

    while (pos < srcLen) {
        if (isspace((unsigned char)src[pos])) { pos++; continue; }
        if (src[pos] == '#') { handlePreprocessor(); continue; }
        if (handleComment()) continue;

        if (isalpha((unsigned char)src[pos]) || src[pos] == '_') { handleWordToken(); continue; }
        if (isdigit((unsigned char)src[pos])) { handleNumberToken(); continue; }
        if (src[pos] == '"') { handleStringToken(); continue; }
        if (src[pos] == '\\') { handleCharToken(); continue; }
        if (handleTwoCharOperator()) continue;

        if (strchr("+-*/%<>=!&|^?:", src[pos])) { handleSingleCharOperator(); continue; }
        if (handleSpecialChar()) continue;

        pos++;
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    printSymbolTable();

    return 0;
}

source.cpp:

#include <iostream>

using namespace std;

// single line comment

/* multi
   line comment */

int main()
{
    int x = 0x1F, count = 5;

    float pi = 3.14;

    char ch = 'A';

    bool done = false;

    x += 2;

    count++;

    if (x <= 10 && count != 0) {
        cout << "ok\n" << ch << endl;
    }

    while (x > 0) {
        x--;
    }

    return 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Output:

```

● kushaal@Kushaals-MacBook-Air Assignment1 % gcc scanner.c -o scanner
● kushaal@Kushaals-MacBook-Air Assignment1 % ./scanner
#include <iostream> - preprocessor directive
using - keyword
namespace - keyword
std - identifier
; - special character
int - keyword
main() - function call
{ - special character
int - keyword
x - identifier
= - assignment operator
0x1F - constant
, - special character
count - identifier
= - assignment operator
5 - constant
; - special character
float - keyword
pi - identifier
= - assignment operator
3.14 - constant
; - special character
char - keyword
ch - identifier
= - assignment operator
'A' - constant
; - special character
bool - keyword
done - identifier
= - assignment operator
false - constant
; - special character
x - identifier
+= - arithmetic assignment operator
2 - integer constant
; - special character
count - identifier
++ - unary operator
; - special character
if - keyword
( - special character
x - identifier
<= - relational operator
10 - integer constant
&& - logical operator
count - identifier
!= - relational operator
0 - integer constant

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

) - special character
{ - special character
cout - identifier
<< - bitwise / stream insertion operator
"ok\n" - string constant
<< - bitwise / stream insertion operator
ch - identifier
<< - bitwise / stream insertion operator
endl - identifier
; - special character
} - special character
while - keyword
( - special character
x - identifier
> - relational operator
0 - integer constant
) - special character
{ - special character
x - identifier
-- - unary operator
; - special character
} - special character
return - keyword
0 - integer constant
; - special character
} - special character

```

Content of Symbol Table

Identifier	Type	No of bytes	Address	Value
x	int	2	1000	0x1F
count	int	2	1002	5
pi	float	4	1004	3.14
ch	char	1	1008	'A'
done	bool	1	1009	false

Learning Outcomes:

- Understood that lexical analysis is the first phase of a compiler and what it is responsible for.
- Learned how the different kinds of tokens in a programming language are recognized and classified.
- Learned how to build and maintain a symbol table that stores the attributes of identifiers.
- Got practical experience in detecting lexical errors such as unterminated comments and strings.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Exp. No: 2

Date: 15-07-2026

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Implementation of Lexical Analyzer using LEX Tool

Aim:

To build a lexical analyzer with the LEX (flex) tool that scans a C++ source program, classifies each token (keywords, identifiers, constants, operators, special characters, comments and function calls), and stores the attributes of every identifier (name, type, number of bytes, value) in a symbol table.

Tools Used:

- Flex
- GCC

Code:

scanner.l

```
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* ----- symbol table ----- */
typedef struct {
    char name[32];
    char type[12];
    int  bytes;
    char value[32];
} SymEntry;

SymEntry table[100];
int symCount = 0;
```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

char activeType[12] = ""; /* type currently being declared on this line */
int lastIdxIndex = -1; /* index of most recently declared identifier */
int awaitingEquals = 0; /* just saw an identifier that could get "= value" */
int awaitingValue = 0; /* just saw "=" after that identifier */

```

```

int sizeOfType(char *t) {
    if (!strcmp(t, "char")) return 1;
    if (!strcmp(t, "bool")) return 1;
    if (!strcmp(t, "float")) return 4;
    if (!strcmp(t, "double")) return 8;
    return 2;
}

```

```

void recordSymbol(char *name) {
    awaitingEquals = 0;
    if (activeType[0] == '\0') { lastIdxIndex = -1; return; }
    strcpy(table[symCount].name, name);
    strcpy(table[symCount].type, activeType);
    table[symCount].bytes = sizeOfType(activeType);
    strcpy(table[symCount].value, "-");
    lastIdxIndex = symCount;
    awaitingEquals = 1;
    symCount++;
}

```

```

void captureValue(char *val) {
    if (awaitingValue && lastIdxIndex >= 0) {

```


Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        strcpy(table[lastIdx].value, val);
    }
    awaitingValue = 0;
    lastIdx = -1;
}

void printSymbolTable(void) {
    printf("\nContent of Symbol Table\n");
    printf("%-15s %-8s %-12s %-10s\n", "Identifier", "Type", "No of bytes", "Value");
    for (int k = 0; k < symCount; k++)
        printf("%-15s %-8s %-12d %-10s\n",
            table[k].name, table[k].type, table[k].bytes, table[k].value);
}

%}

%option nounput noinput
%x COMMENT

DIGIT    [0-9]
LETTER   [a-zA-Z_]
HEX      [0-9a-fA-F]

%%

"/".*    { /* single line comment: ignore */ }

"/*"     { BEGIN(COMMENT); }

<COMMENT>"*/" { BEGIN(INITIAL); }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

<COMMENT>\n                { /* stay in comment */ }
<COMMENT>.                  { /* consume comment body */ }
<COMMENT><<EOF>>            {
                                printf("Error: unterminated comment\n");
                                yyterminate();
                                }

```

```

"auto"|"break"|"case"|"catch"|"class"|"const"|"constexpr"|"continue"|"default"|"delete"|"do"|"else"|"
enum"|"explicit"|"extern"|"for"|"friend"|"goto"|"if"|"inline"|"mutable"|"namespace"|"new"|"nullptr"|"o
perator"|"private"|"protected"|"public"|"register"|"return"|"sizeof"|"static"|"struct"|"switch"|"templat
e"|"this"|"throw"|"try"|"typedef"|"typename"|"union"|"unsigned"|"using"|"virtual"|"void"|"volatile"|"w
hile"|"signed" {
                                printf("%s - keyword\n", yytext);
                                activeType[0] = '\0';
                                }

```

```

"int"|"char"|"float"|"double"|"long"|"short"|"bool" {
                                printf("%s - keyword\n", yytext);
                                strcpy(activeType, yytext);
                                }

```

```

^[ \t]*#[^\n]*                { printf("%s - preprocessor directive\n", yytext + strspn(yytext, " \t")); }

```

```

"std::?"cout"                  { printf("%s - output stream (function call)\n", yytext); }

```

```

"std::?"cin"                   { printf("%s - input stream (function call)\n", yytext); }

```

```

"std::?"endl"                  { printf("%s - stream manipulator\n", yytext); }

```

```

"printf"[ \t]*("[^"]*)"         { printf("%s - function call\n", yytext); }

```

```

"scanf"[ \t]*("[^"]*)"          { printf("%s - function call\n", yytext); }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

"getch"[\t]*("[^"]*)" { printf("%s - function call\n", yytext); }

"clrscr"[\t]*("[^"]*)" { printf("%s - function call\n", yytext); }

"main"[\t]*("[^"]*)" { printf("%s - function call\n", yytext); }

"true"|"false" { printf("%s - boolean constant\n", yytext); captureValue(yytext); }

{LETTER}{(LETTER){DIGIT}}* {
printf("%s - identifier\n", yytext);
recordSymbol(yytext);
}

"0"[xX]{HEX}+ { printf("%s - hexadecimal constant\n", yytext);
captureValue(yytext); }

{DIGIT}+"."{DIGIT}+ { printf("%s - double constant\n", yytext); captureValue(yytext); }

{DIGIT}+ { printf("%s - integer constant\n", yytext); captureValue(yytext); }

\("[^"\n"]*\)" { printf("%s - string constant\n", yytext); captureValue(yytext); }

\('[^"\\\n\.\.]*\)' { printf("%s - character constant\n", yytext); captureValue(yytext); }

"+="|"-"|"*"|"/"|"%"= " { printf("%s - arithmetic assignment operator\n", yytext);
awaitingEquals = 0; awaitingValue = 0; }

"++"|"--" { printf("%s - unary operator\n", yytext); }

"&&" { printf("&& - logical operator\n"); }

"||" { printf("|| - logical operator\n"); }

"==" { printf("== - relational operator\n"); }

"!=" { printf("!= - relational operator\n"); }

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

"<="      { printf("<= - relational operator\n"); }
">="      { printf(">= - relational operator\n"); }
"<<"      { printf("<< - bitwise / stream insertion operator\n"); }
">>"      { printf(">> - bitwise / stream extraction operator\n"); }
"::"      { printf(":: - scope resolution operator\n"); }
"->"      { printf("-> - member access operator\n"); }

[+|-|*|/]  { printf("%c - arithmetic operator\n", yytext[0]); }
[<|>]     { printf("%c - relational operator\n", yytext[0]); }
"="        {
    printf("= - assignment operator\n");
    if (awaitingEquals) { awaitingValue = 1; awaitingEquals = 0; }
}
"!"        { printf("! - logical operator\n"); }
[&|^]      { printf("%c - bitwise operator\n", yytext[0]); }

[;|.|\\|[]|{}|?:] {
    printf("%c - special character\n", yytext[0]);
    if (yytext[0] == ';') activeType[0] = '\0';
}

[ \t\n]+   { /* skip whitespace */ }

.           { /* skip anything unrecognized */ }

%%

int yywrap(void) { return 1; }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
int main(void) {
    yylex();
    printSymbolTable();
    return 0;
}
```

sample.cpp:

```
{
    int a=10,b=20;
    if(a>b)
        cout << "a is greater" << endl;
    else
        std::cout << "b is greater" << std::endl;
}
```

Output:

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

kushaal@Kushaals-MacBook-Air Assignment 2 % ./scanner < sample.cpp
{ - special character
int - keyword
a - identifier
= - assignment operator
10 - integer constant
, - special character
b - identifier
= - assignment operator
20 - integer constant
; - special character
if - keyword
( - special character
a - identifier
> - relational operator
b - identifier
) - special character
cout - output stream (function call)
<< - bitwise / stream insertion operator
"a is greater" - string constant
<< - bitwise / stream insertion operator
endl - stream manipulator
; - special character
else - keyword
std::cout - output stream (function call)
<< - bitwise / stream insertion operator
"b is greater" - string constant
<< - bitwise / stream insertion operator
std::endl - stream manipulator
; - special character
} - special character

```

Content of Symbol Table

Identifier	Type	No of bytes	Value
a	int	2	10
b	int	2	20

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Question Answers

Q1. How does your scanner handle multi-line comments that are not terminated correctly? How does it report errors in such cases?

The scanner handles comments with a Flex exclusive start condition named COMMENT. On reading `/*`, it enters this state through `BEGIN(COMMENT)` and silently discards every character until it meets the closing `*/`, where it returns to INITIAL. If the file ends while the scanner is still in the COMMENT state, meaning no `*/` was found, the `<COMMENT><<EOF>>` rule fires, prints Error: unterminated comment and stops the scan through `yyterminate()`. This was tested with a file containing `/*` this comment never ends and no closing tag: the valid tokens before it were processed correctly and then exactly that error message was printed.

Q2. Construct symbol table for storing the identifier information

The symbol table is an array of `SymEntry` structures, each holding the name, type, size in bytes (derived from the type: `char=1`, `bool=1`, `float=4`, `double=8`, otherwise 2) and value. A global "active type" remembers the last type keyword read (for example `int`), and the next identifier is stored under that type. If an `=` follows the identifier, the next constant token (integer, hex, double, char or string) is saved as its value. The table is printed once, after the whole file has been scanned, in this format:

Identifier	Type	No of bytes	Value
a	int	2	10
b	int	2	20

9. Learning Outcomes

- Learned to write lexical specifications with regular expressions in Flex.
- Understood Flex's longest-match rule and how the order and specificity of patterns affect tokenization (a bug where `if(a>b)` was caught by a generic function-call pattern was fixed by restricting that pattern to named functions).
- Used exclusive start conditions (`%x`) to handle multi-line comments correctly and to detect unterminated ones through `<<EOF>>`.
- Built a symbol table that records assigned values as well as declarations, using small state flags to connect an identifier with the `=` and constant that follow it.

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

Exp. No: 3

Date: 24-07-2026

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Implementation of Recursive Decent Parser

Aim

To write a recursive descent parser for the given grammar, after removing its left recursion.

Grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow (E) \mid id$

Tools Used

- GCC

Code (rdp.c)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
const char *input;
```

```
int i = 0;
```

```
int error = 0;
```

```
void E();
```

```
void E_prime();
```


Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```
void T();
```

```
void T_prime();
```

```
void F();
```

```
void match(char expected) {
```

```
    if (input[i] == expected) {
```

```
        i++;
```

```
    } else {
```

```
        error = 1;
```

```
    }
```

```
}
```

```
void match_id() {
```

```
    if (input[i] == 'i' && input[i + 1] == 'd') {
```

```
        i += 2;
```

```
    } else {
```

```
        error = 1;
```

```
    }
```

```
}
```

```
void E() {
```

```
    T();
```

```
    E_prime();
```

```
}
```

```
void E_prime() {
```

```
    if (input[i] == '+') {
```

```
        match('+');
```

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```

    T();
    E_prime();
}

}

void T() {
    F();
    T_prime();
}

void T_prime() {
    if (input[i] == '*') {
        match('*');
        F();
        T_prime();
    }
}

void F() {
    if (input[i] == '(') {
        match('(');
        E();
        match(')');
    } else if (input[i] == 'i' && input[i + 1] == 'd') {
        match_id();
    } else {
        error = 1;
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

}

```
void parse_string(const char *str) {
    input = str;
    i = 0;
    error = 0;
    printf("Parsing string: \"%s\\n\", input);

    E();
    if (error == 0 && input[i] == '\\0') {
        printf("Result: SUCCESS (String successfully parsed!)\\n\\n");
    } else {
        printf("Result: REJECTED (Syntax Error detected!)\\n\\n");
    }
}
```

```
int main() {
    parse_string("id+id*id");
    parse_string("(id+id*id");
    parse_string("id-id");
    return 0;
}
```

Output:

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```

● kushaal@Kushaals-MacBook-Air Compiler-Design % cd "Assignment 3"
● kushaal@Kushaals-MacBook-Air Assignment 3 % gcc rec.c -o rec
● kushaal@Kushaals-MacBook-Air Assignment 3 % ./rec
Parsing string: "id+id*id"
Result: SUCCESS (String successfully parsed!)

Parsing string: "(id+id*id"
Result: REJECTED (Syntax Error detected!)

Parsing string: "id-id"
Result: REJECTED (Syntax Error detected!)

```

Learning Outcomes

- Learnt to turn each grammar rule into a recursive function so that top-down parsing is carried out in C.
- Learnt to handle operator precedence, match tokens, and detect syntax errors while the program runs.

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

Exp. No: 4

Date: 31-07-2026

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Implementation of Desk Calculator using Yacc Tool

Aim

To write a Lex program that recognizes the tokens needed by a Yacc parser, and a Yacc desk calculator that evaluates arithmetic expressions using +, -, *, /, ^, (and), with the correct precedence and associativity for each operator.

Tools Used

Flex

Bison

GCC

Code (ex.I)

```
%{
#include "y.tab.h"
%}

%%

[0-9]+(\.[0-9]+)? { yylval.dval = atof(yytext); return NUMBER; }

[ \t]           ;

\n              return '\n';

[+ \- * / ^()]   return yytext[0];

.               ;

%%
```

```
int yywrap(void) {
```

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```

    return 1;
}

```

Code (ex.y)

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int yylex(void);
void yyerror(char *s);
}%

%union {
    double dval;
}

%token <dval> NUMBER
%type <dval> expr

%left '+' '-'
%left '*' '/'
%right '^'

%start program

%%

```

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

program : program line

|
;

line : expr '\n' { printf("Result = %g\n", \$1); }

| '\n'
| error '\n' { yyerror; }
;

expr : expr '+' expr { \$\$ = \$1 + \$3; }

| expr '-' expr { \$\$ = \$1 - \$3; }
| expr '*' expr { \$\$ = \$1 * \$3; }
| expr '/' expr { \$\$ = \$1 / \$3; }
| expr '^' expr { \$\$ = pow(\$1, \$3); }
| '(' expr ')' { \$\$ = \$2; }
| NUMBER { \$\$ = \$1; }
;

%%

void yyerror(char *s) {

printf("%s\n", s);
}

int main(void) {

printf("Enter expression:\n");
yyparse();
return 0;

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

}

Sample Inputs (Test Cases)

3+9

3+9*6

(3+4)*7

(3-4)+(7*6)

5/7+2

4^2^1

(2^3)^2

Commands Used

lex ex.l

yacc -d ex.y

cc lex.yy.c y.tab.c -lfl -lm -o calc

./calc

Output:

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```

kushaal@Kushaals-MacBook-Air Assignment 4 % yacc -d ex.y
kushaal@Kushaals-MacBook-Air Assignment 4 % gcc lex.yy.c y.tab.c -o calc -lm
kushaal@Kushaals-MacBook-Air Assignment 4 % ./calc
Enter expression:
2+3*4
Result = 14

5+2
Result = 7

9/3+2^4
Result = 19

```

Learning Outcomes

Understood how Yacc resolves ambiguity in an arithmetic grammar through precedence and associativity declarations (%left, %right).

Learned that the grammar needs a left-recursive start production such as program : program line | ; so the parser keeps reading lines instead of stopping after the first.

Learned how Lex and Yacc exchange tokens through a %union-typed yylval/YYSTYPE, and why that type has to be the same in the generated lexer and parser.

Used Yacc's error token and yyerror to recover from a bad line and keep the calculator running.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 5

Date: 07-08-2026

Implementation of Syntax Checker using Lex and Yacc Tools

Aim:

To build a syntax checker with Lex and Yacc that recognizes declaration, assignment, conditional and looping statements, using a suitable grammar for each. It works on C++ source: apart from the C-like statements it accepts the bool type, cout << and cin >> statements, return, #include and using namespace lines, and an int main() wrapper.

Tools Used:

- Flex, the lexical analyzer generator
- Bison, the parser generator (Yacc)
- GCC

Code:

syntax.l

```
%{
#include "y.tab.h"
%}

%option yylineno noyywrap nounput noinput

DIGIT  [0-9]
LETTER [a-zA-Z_]

%%

"//"*. *           ;
"/"*(\[^\*]\|\\*\[^\*/\])*\*+/"          ;

^\[ \t]*"#\[^\n]*           ;
"using"[ \t]+"namespace"[ \t]+{LETTER}({LETTER}|{DIGIT})*[ \t]*";"    ;

"int"|"float"|"char"|"double"|"bool" { return TYPE; }
"return"                             { return RETURN; }
"std::"?"cout"                       { return COUT; }
"std::"?"cin"                        { return CIN; }
"std::"?"endl"                      { return ENDL; }
"true"|"false"                      { return NUM; }
"if"                                 { return IF; }
"else"                               { return ELSE; }
"while"                             { return WHILE; }
"do"                                 { return DO; }
"for"                                { return FOR; }

{LETTER}({LETTER}|{DIGIT})*         { return ID; }
{DIGIT}+(\.{DIGIT})?               { return NUM; }
\"([^\\"\\n]|\\.)*\"                 { return STR; }
\'([^\'\\]|\\.)*\'                   { return NUM; }

"++"                                { return INC; }
"--"                                { return DEC; }
"<<"                                { return SHL; }
">>"                                { return SHR; }
```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

"&&"      { return AND; }
"||"      { return OR; }
"=="|"!=" { return EQOP; }
"<="|">="|"<"|>" { return RELOP; }
"! "      { return NOT; }
"="       { return '='; }
[+\-*/%]  { return yytext[0]; }
[(){};,]  { return yytext[0]; }

[ \t\r\n] ;
.         { return yytext[0]; }

%%

```

syntax.y

```

%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *s);

extern int  yylineno;
extern char *yytext;
%}

%token TYPE IF ELSE WHILE DO FOR ID NUM STR
%token RETURN COUT CIN ENDL SHL SHR
%token INC DEC AND OR EQOP RELOP NOT

%left OR
%left AND
%left EQOP
%left RELOP
%left '+' '-'
%left '*' '/' '%'
%right NOT UMINUS

%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE

%%

program      : stmt_list
              | function
              ;

/* int main() { ... }      (optionally int main(void) ) */
function     : TYPE ID '(' ')' block
              | TYPE ID '(' TYPE ')' block
              ;

stmt_list    : stmt_list stmt
              | stmt
              ;

stmt         : decl_stmt
              | assign_stmt
              | if_stmt
              | while_stmt
              | do_stmt
              | for_stmt
              | io_stmt
              | return_stmt
              | block
              | ';'

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

;

/* Output / input statements: cout << a << "text" << endl; cin >> a; */
io_stmt : COUT out_list ';'
        | CIN in_list ';'
        ;

out_list : SHL out_item
        | out_list SHL out_item
        ;

out_item : STR
        | ENDL
        | expr
        ;

in_list : SHR ID
        | in_list SHR ID
        ;

return_stmt : RETURN expr ';'
            | RETURN ';'
            ;

block : '{' stmt_list '}'
      | '{' '}'
      ;

/* Declaration statement: int i, a = 5; */
decl_stmt : TYPE decl_list ';'
          ;

decl_list : decl_list ',' declarator
          | declarator
          ;

declarator : ID
          | ID '=' expr
          ;

/* Assignment statement: i = i + 1; i++; */
assign_stmt : simple_stmt ';'
            ;

simple_stmt : ID '=' expr
           | ID INC
           | ID DEC
           ;

/* Conditional statements: if, if-else */
if_stmt : IF '(' expr ')' stmt %prec LOWER_THAN_ELSE
        | IF '(' expr ')' stmt ELSE stmt
        ;

/* Looping statements: while, do-while, for */
while_stmt : WHILE '(' expr ')' stmt
           ;

do_stmt : DO stmt WHILE '(' expr ')' ';'
        ;

for_stmt : FOR '(' simple_stmt ';' expr ';' simple_stmt ')' stmt
        ;

expr : expr OR expr
     | expr AND expr
     | expr EQOP expr

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    | expr RELOP expr
    | expr '+' expr
    | expr '-' expr
    | expr '*' expr
    | expr '/' expr
    | expr '%' expr
    | NOT expr
    | '-' expr %prec UMINUS
    | '(' expr ')'
    | ID
    | NUM
    ;

%%

void yyerror(const char *s) {
    printf("Syntax Error at line %d near '%s'\n", yylineno, yytext);
}

int main(void) {
    if (yyparse() == 0)
        printf("Syntactically correct\n");
    return 0;
}

```

Input (input.txt)

```

int i, a;
i = 0;
a=5;
while (i < 10)
{
    if ( i<a)
    i = i + 1;
    else
    i = i - 1;
}

```

Commands Used

```

lex syntax.l
yacc -d syntax.y
cc lex.yy.c y.tab.c -lfl -o syncheck
./syncheck < input.txt

```

Output:

```

● kushaal@Kushaals-MacBook-Air Assignment 5 % flex syntax.l
● kushaal@Kushaals-MacBook-Air Assignment 5 % yacc -d syntax.y
● kushaal@Kushaals-MacBook-Air Assignment 5 % gcc lex.yy.c y.tab.c -o syncheck
● kushaal@Kushaals-MacBook-Air Assignment 5 % ./syncheck < input.txt
Syntactically correct

```

Result for the given input

The grammar has no conflicts: operator precedence and associativity are set with %left / %right and the dangling else is settled with %prec LOWER_THAN_ELSE, so yacc reports no shift/reduce conflicts.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Test Cases:

```

● kushaal@Kushaals-MacBook-Air Assignment 5 % cat input2.txt
int n = 5, sum, i;
float avg;
sum = 0;
for (i = 0; i < n; i++)
{
    if (i % 2 == 0 && i != 4)
        sum = sum + i * 2;
    else if (!(i > 3))
        sum = sum - 1;
}
do
{
    n--;
} while (n >= 0 || sum > 100);
avg = sum / (n + 1);
● kushaal@Kushaals-MacBook-Air Assignment 5 % ./syncheck < input2.txt
Syntactically correct
● kushaal@Kushaals-MacBook-Air Assignment 5 % cat input3.txt
int i, a;
i = 0
a = 5;
while (i < 10)
{
    i = i + 1;
}
● kushaal@Kushaals-MacBook-Air Assignment 5 % ./syncheck < input3.txt
Syntax Error at line 3 near 'a'
● kushaal@Kushaals-MacBook-Air Assignment 5 % cat input4.txt
int i;
i = 0;
while (i < 10
{
    i = i + 1;
}
● kushaal@Kushaals-MacBook-Air Assignment 5 % ./syncheck < input4.txt
Syntax Error at line 4 near '{'

```

Further test cases: one valid program and two with syntax errors

input2.txt contains every kind of statement: declarations with initial values, for, if / else if, do-while, logical operators and increment / decrement. It is accepted as Syntactically correct.

input3.txt is missing the semicolon after `i = 0`, and input4.txt is missing the closing bracket of the while condition. The checker prints the line number and the token at which parsing can no longer continue.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

C++ test cases (input5.txt and input6.txt):

```

input5.txt:
#include <iostream>
using namespace std;

int main()
{
    int n = 5, sum = 0, i;
    bool done = false;
    cin >> n;
    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0 && i != 4)
            sum = sum + i * 2;
        else
            sum = sum - 1;
    }
    cout << "Sum = " << sum << endl;
    return 0;
}
Result: Syntactically correct

input6.txt (missing semicolon after endl):
#include <iostream>
using namespace std;

int main()
{
    int a = 5;
    cout << "a is " << a << endl
    return 0;
}
Result: Syntax Error at line 8 near 'return'

```

Learning Outcomes:

- Learned to write a grammar for declaration, assignment, conditional (if-else) and looping (while, do-while, for) statements of a C / C++ like language.
- Understood how precedence declarations (%left, %right) and %prec eliminate shift/reduce conflicts, including the dangling else.
- Understood how Yacc generates a parser from BNF-style productions and reports syntax errors through yyerror, with yylineno giving the line number.
- Gained experience combining Lex tokens with a Yacc grammar to validate the structure of a small program.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 6

Date: 14-08-2026

Generation of Intermediate Code using Lex and Yacc Tools

Aim:

To build an intermediate code generator with Lex and Yacc that produces three address code for assignment statements and boolean expressions, using syntax directed translation rules.

Tools Used:

- Flex
- Bison
- GCC

Code:

tac.l

```
%{
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include "y.tab.h"

char stmt_text[256];          /* source text of the statement just read */
static char srcline[256];    /* characters of the statement being read */

#define YY_USER_ACTION  strncat(srcline, yytext, sizeof(srcline) - strlen(srcline) - 1);

/* Move srcline into stmt_text (without the terminator) and start afresh */
static void end_statement(void)
{
    char *s = srcline;
    int n;
    while (isspace((unsigned char)*s)) s++;
    strcpy(stmt_text, s);
    n = strlen(stmt_text);
    while (n > 0 && (isspace((unsigned char)stmt_text[n-1]) || stmt_text[n-1] == ';'))
        stmt_text[--n] = '\0';
    srcline[0] = '\0';
}

static int pending(void)
{
    char *s;
    for (s = srcline; *s; s++)
        if (!isspace((unsigned char)*s)) return 1;
    return 0;
}
}%

%option yylineno

DIGIT  [0-9]
LETTER [a-zA-Z_]

%%

"or"|"||" { return OR; }
```


Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

"and"|"&&"          { return AND; }
"not"|"!"            { return NOT; }

{LETTER}({LETTER}|{DIGIT})*
{DIGIT}+(\.{DIGIT}+)? { yylval.str = strdup(yytext); return ID; }
                      { yylval.str = strdup(yytext); return NUM; }

":="|"="            { return ASSIGN; }
"<="|">="|"=="|"!="|"<"|">"
[+ \- * / % ()]      { return yytext[0]; }

[ \n;]               { end_statement(); return EOL; }
[ \t \r ]+           ;
.                    { return yytext[0]; }

<<EOF>>              { if (pending()) { end_statement(); return EOL; }
                      return 0; }

%%

int yywrap(void)
{
    return 1;
}

```

tac.y

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>

int yylex(void);
void yyerror(const char *s);

extern int yylineno;
extern char stmt_text[];

#define BASE 100          /* number given to the first instruction */
#define MAXCODE 100

static char code[MAXCODE][80]; /* three address code of the current statement */
static int ncode = 0;          /* instructions generated so far */
static int ntemp = 0;          /* temporaries used so far */
static int hasBool = 0;        /* statement contains a boolean expression */

static char *newtemp(void)
{
    char buf[16];
    sprintf(buf, "t%d", ++ntemp);
    return strdup(buf);
}

static void emit(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(code[ncode++], sizeof(code[0]), fmt, ap);
    va_end(ap);
}

static void reset(void)
{
    ncode = ntemp = hasBool = 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

/* Print the code of one statement: numbered from 100 only if it has booleans */
static void flush_statement(void)
{
    int k;
    printf("%s\n", stmt_text);
    for (k = 0; k < (int)strlen(stmt_text); k++) putchar('-');
    putchar('\n');
    for (k = 0; k < ncode; k++) {
        if (hasBool) printf("%d: %s\n", BASE + k, code[k]);
        else          printf("%s\n", code[k]);
    }
    putchar('\n');
    reset();
}

/* result = x op y for arithmetic operators */
static char *arith(char *x, char op, char *y)
{
    char *t = newtemp();
    emit("%s = %s %c %s", t, x, op, y);
    return t;
}

/* result = x relop y : four instructions giving the value 0 or 1 */
static char *relational(char *x, char *op, char *y)
{
    char *t = newtemp();
    int a = BASE + ncode; /* number of the 'if' instruction */
    hasBool = 1;
    emit("if %s %s %s goto %d", x, op, y, a + 3);
    emit("%s := 0", t);
    emit("goto %d", a + 4);
    emit("%s := 1", t);
    return t;
}

/* result = x and y / x or y */
static char *logical(char *x, const char *op, char *y)
{
    char *t = newtemp();
    hasBool = 1;
    emit("%s := %s %s %s", t, x, op, y);
    return t;
}
%}

%union { char *str; }

%token <str> ID NUM RELOP
%token ASSIGN OR AND NOT EOL

%type <str> expr

%left OR
%left AND
%right NOT
%nonassoc RELOP
%left '+' '-'
%left '*' '/' '%'
%right UMINUS

%%

program : program stmt
        |
        ;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

stmt      : ID ASSIGN expr EOL      { emit("%s = %s", $1, $3); flush_statement(); }
          | EOL
          | error EOL               { reset(); yyerrorok; }
          ;

expr       : expr OR expr           { $$ = logical($1, "or", $3); }
          | expr AND expr          { $$ = logical($1, "and", $3); }
          | NOT expr               { $$ = newtemp(); hasBool = 1;
                                   emit("%s := not %s", $$, $2); }
          | expr RELOP expr        { $$ = relational($1, $2, $3); }
          | expr '+' expr          { $$ = arith($1, '+', $3); }
          | expr '-' expr          { $$ = arith($1, '-', $3); }
          | expr '*' expr          { $$ = arith($1, '*', $3); }
          | expr '/' expr          { $$ = arith($1, '/', $3); }
          | expr '%' expr          { $$ = arith($1, '%', $3); }
          | '-' expr %prec UMINUS { $$ = newtemp(); emit("%s = -%s", $$, $2); }
          | '(' expr ')'           { $$ = $2; }
          | ID                     { $$ = $1; }
          | NUM                    { $$ = $1; }
          ;

%%

void yyerror(const char *s)
{
    printf("Syntax error at line %d\n\n", yylineno);
}

int main(void)
{
    yyparse();
    return 0;
}

```

Input (input.txt)

```

a := b * -c + b* -c
x:= a<b or c<d and e<f

```

Commands Used

```

lex tac.l
yacc -d tac.y
cc lex.yy.c y.tab.c -lfl -o tac
./tac < input.txt

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Output:

```

● kushaal@Kushaals-MacBook-Air Assignment 6 % flex tac.l
● kushaal@Kushaals-MacBook-Air Assignment 6 % yacc -d tac.y
● kushaal@Kushaals-MacBook-Air Assignment 6 % gcc lex.yy.c y.tab.c -o tac
● kushaal@Kushaals-MacBook-Air Assignment 6 % ./tac < input.txt
a := b * -c + b* -c
-----
t1 = -c
t2 = b * t1
t3 = -c
t4 = b * t3
t5 = t2 + t4
a = t5

x:= a<b or c<d and e<f
-----
100: if a < b goto 103
101: t1 := 0
102: goto 104
103: t1 := 1
104: if c < d goto 107
105: t2 := 0
106: goto 108
107: t2 := 1
108: if e < f goto 111
109: t3 := 0
110: goto 112
111: t3 := 1
112: t4 := t2 and t3
113: t5 := t1 or t4
114: x = t5

```

Three address code generated for the given input

The temporaries t1, t2, ... restart for each statement. A statement containing a boolean expression is numbered from 100, and a relational expression $a < b$ becomes four instructions: if $a < b$ goto (n+3), $t := 0$, goto (n+4), $t := 1$. Operator priority from lowest to highest is or, and, not, relational, arithmetic, so in $a < b$ or $c < d$ and $e < f$ the and is evaluated before the or.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Test Cases:

```

• kushaal@Kushaals-MacBook-Air Assignment 6 % cat input2.txt
y := (a + b) * (c - d) / 2
z := not a < b and c >= d
p := a + b * c < d - 1
q := a +* b
• kushaal@Kushaals-MacBook-Air Assignment 6 % ./tac < input2.txt
y := (a + b) * (c - d) / 2
-----
t1 = a + b
t2 = c - d
t3 = t1 * t2
t4 = t3 / 2
y = t4

z := not a < b and c >= d
-----
100: if a < b goto 103
101: t1 := 0
102: goto 104
103: t1 := 1
104: t2 := not t1
105: if c >= d goto 108
106: t3 := 0
107: goto 109
108: t3 := 1
109: t4 := t2 and t3
110: z = t4

p := a + b * c < d - 1
-----
100: t1 = b * c
101: t2 = a + t1
102: t3 = d - 1
103: if t2 < t3 goto 106
104: t4 := 0
105: goto 107
106: t4 := 1
107: p = t4

Syntax error at line 4

```

More test cases (the last statement contains a syntax error)

Learning Outcomes:

- Learned how syntax directed translation attaches semantic actions to grammar rules, so three address code is produced while the input is parsed.
- Understood how the attributes (\$\$, \$1, \$3) carry the name of the temporary holding each sub-expression's value, and how new temporaries are created.
- Learned to translate relational expressions into conditional jumps with instruction numbers, and to combine them using and / or / not.
- Used precedence declarations to give unary minus, arithmetic, relational and logical operators the right priority, and the yacc error token to recover from a bad line.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 7

Date: 21-08-2026

Implementation of Code Optimization Techniques

Aim:

To implement the code optimization techniques of constant folding, algebraic identities, strength reduction and dead code elimination on a sequence of three address code statements.

Tools Used:

- GCC

Code:

optimizer.c

```

/*
 * Code optimization on Three Address Code (straight-line block)
 *
 * 1. Constant folding      5 * 3      -> 15    (with constant propagation)
 * 2. Algebraic identities  x+0, x*1    -> x
 * 3. Strength reduction    x**2, x*2    -> x*x, x+x
 * 4. Dead code elimination assignments whose result is never used
 *
 * Input : one instruction per line      dest = a          (copy)
 *                                           dest = a op b      (op: + - * / % ** << >>)
 *           optional last line          live: t1, t3 ...    (variables needed after the block)
 * Usage : ./optimizer < input.txt      or      ./optimizer input.txt
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <ctype.h>

#define MAXI 200          /* maximum number of instructions */
#define MAXN 32           /* maximum length of a name      */
#define MAXV 600         /* maximum size of a variable set */
#define LINE 128         /* buffer for one printed instruction */

typedef struct {
    char dest[MAXN];      /* result                        */
    char a[MAXN];         /* first operand                 */
    char b[MAXN];         /* second operand (binary only) */
    char op[3];           /* operator; "" means a plain copy */
    int dead;             /* removed by the optimizer      */
} Instr;

static Instr prog[MAXI];
static int n = 0;        /* number of instructions        */
static char liveOut[MAXV][MAXN]; /* variables live after the block */
static int nLive = 0, haveLive = 0;

/* ----- helpers ----- */

static int isnum(const char *s)
{
    if (*s == '-') s++;
    if (!*s) return 0;
    while (*s) if (!isdigit((unsigned char)*s++)) return 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    return 1;
}

static int isvar(const char *s)
{
    return s[0] && !isnum(s);
}

static void show(const Instr *in, char *buf)
{
    if (in->op[0]) sprintf(buf, "%s = %s %s %s", in->dest, in->a, in->op, in->b);
    else          sprintf(buf, "%s = %s", in->dest, in->a);
}

static void make_copy(Instr *in, const char *value)
{
    char tmp[MAXN];
    strcpy(tmp, value);          /* value may point inside the instruction */
    strcpy(in->a, tmp);
    in->op[0] = in->b[0] = '\0';
}

static int power_of_two(const char *s)          /* returns k if s == 2^k, else -1 */
{
    long long v;
    int k = 0;
    if (!isnum(s)) return -1;
    v = atoll(s);
    if (v < 1 || (v & (v - 1))) return -1;
    while (v > 1) { v >>= 1; k++; }
    return k;
}

/* ----- logging */

static int round_no = 1, round_shown = 0;
static int pass_shown[5];

static void note(int pass, const char *name, const char *before, const char *after,
                const char *why)
{
    if (round_no > 1 && !round_shown) {
        printf("\n----- Round %d ----- \n", round_no);
        round_shown = 1;
    }
    if (!pass_shown[pass]) {
        printf("\n[%d] %s\n", pass, name);
        pass_shown[pass] = 1;
    }
    printf("    %-18s ==>  %-18s (%s)\n", before, after ? after : "removed", why);
}

static void no_change(int pass, const char *name)
{
    if (round_no == 1 && !pass_shown[pass]) {
        printf("\n[%d] %s\n    (no change)\n", pass, name);
        pass_shown[pass] = 1;
    }
}

/* ----- 1. constant folding + propagation */

static int evaluate(long long x, const char *op, long long y, long long *r)
{
    long long p = 1;
    if (!strcmp(op, "+")) *r = x + y;
    else if (!strcmp(op, "-")) *r = x - y;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

else if (!strcmp(op, "*")) *r = x * y;
else if (!strcmp(op, "/")) { if (y == 0) return 0; *r = x / y; }
else if (!strcmp(op, "%")) { if (y == 0) return 0; *r = x % y; }
else if (!strcmp(op, "<<")) { if (y < 0 || y > 62) return 0; *r = x << y; }
else if (!strcmp(op, ">>")) { if (y < 0 || y > 62) return 0; *r = x >> y; }
else if (!strcmp(op, "**")) { if (y < 0 || y > 62) return 0; while (y-- > 0) p *= x; *r = p; }
else return 0;
return 1;
}

static int constant_folding(void)
{
    const char *title = "Constant Folding (with constant propagation)";
    static char kname[MAXI][MAXN], kval[MAXI][MAXN]; /* variables known to be constant */
    int nk = 0, i, k, changes = 0;
    char before[LINE], after[LINE], val[MAXN];

    for (i = 0; i < n; i++) {
        Instr *in = &prog[i];
        int propagated = 0, folded = 0;
        if (in->dead) continue;
        show(in, before);

        /* constant propagation: replace a variable by its known constant */
        for (k = 0; k < nk; k++) {
            if (!kname[k][0]) continue;
            if (!strcmp(in->a, kname[k])) { strcpy(in->a, kval[k]); propagated = 1; }
            if (in->op[0] && !strcmp(in->b, kname[k])) { strcpy(in->b, kval[k]); propagated = 1; }
        }
        /* constant folding: both operands are constants */
        if (in->op[0] && isnum(in->a) && isnum(in->b)) {
            long long r;
            if (evaluate(atoll(in->a), in->op, atoll(in->b), &r)) {
                sprintf(val, "%lld", r);
                make_copy(in, val);
                folded = 1;
            }
        }
        show(in, after);
        if (strcmp(before, after)) {
            note(1, title, before, after,
                folded ? (propagated ? "constants propagated, then evaluated"
                    : "evaluated at compile time")
                    : "constant propagated");
            changes++;
        }

        /* update the table of known constants */
        for (k = 0; k < nk; k++)
            if (!strcmp(kname[k], in->dest)) kname[k][0] = '\0';
        if (!in->op[0] && isnum(in->a)) {
            for (k = 0; k < nk && kname[k][0]; k++) ;
            if (k == nk) nk++;
            strcpy(kname[k], in->dest);
            strcpy(kval[k], in->a);
        }
    }
    if (!changes) no_change(1, title);
    return changes;
}

/* ----- 2. algebraic identities */

static int algebraic_identities(void)
{
    const char *title = "Algebraic Identities";
    int i, changes = 0;

```


Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

char before[LINE], after[LINE], keep[MAXN];

for (i = 0; i < n; i++) {
    Instr *in = &prog[i];
    const char *op = in->op, *why = NULL;
    int a0, b0, a1, b1;
    if (in->dead || !op[0]) continue;
    show(in, before);
    a0 = !strcmp(in->a, "0"); b0 = !strcmp(in->b, "0");
    a1 = !strcmp(in->a, "1"); b1 = !strcmp(in->b, "1");
    strcpy(keep, in->a); /* value kept when x op e = x */

    if (!strcmp(op, "+") && b0) { make_copy(in, keep); why = "x + 0 = x"; }
    else if (!strcmp(op, "+") && a0) { make_copy(in, in->b); why = "0 + x = x"; }
    else if (!strcmp(op, "-") && b0) { make_copy(in, keep); why = "x - 0 = x"; }
    else if (!strcmp(op, "-") && isvar(in->a) && !strcmp(in->a, in->b))
        { make_copy(in, "0"); why = "x - x = 0"; }
    else if (!strcmp(op, "*") && b1) { make_copy(in, keep); why = "x * 1 = x"; }
    else if (!strcmp(op, "*") && a1) { make_copy(in, in->b); why = "1 * x = x"; }
    else if (!strcmp(op, "*") && (a0 || b0))
        { make_copy(in, "0"); why = "x * 0 = 0"; }
    else if (!strcmp(op, "/") && b1) { make_copy(in, keep); why = "x / 1 = x"; }
    else if (!strcmp(op, "%") && b1) { make_copy(in, "0"); why = "x % 1 = 0"; }
    else if (!strcmp(op, "**") && b1) { make_copy(in, keep); why = "x ** 1 = x"; }
    else if (!strcmp(op, "**") && b0) { make_copy(in, "1"); why = "x ** 0 = 1"; }
    else if ((!strcmp(op, "<<") || !strcmp(op, ">>")) && b0)
        { make_copy(in, keep); why = "x shifted by 0 = x"; }

    if (why) {
        show(in, after);
        note(2, title, before, after, why);
        changes++;
    }
}
if (!changes) no_change(2, title);
return changes;
}

/* ----- 3. strength reduction */

static int strength_reduction(void)
{
    const char *title = "Strength Reduction";
    int i, k, changes = 0;
    char before[LINE], after[LINE], v[MAXN], num[MAXN];

    for (i = 0; i < n; i++) {
        Instr *in = &prog[i];
        const char *why = NULL;
        if (in->dead || !in->op[0]) continue;
        show(in, before);

        if (!strcmp(in->op, "**") && !strcmp(in->b, "2") && isvar(in->a)) {
            strcpy(in->b, in->a);
            strcpy(in->op, "*");
            why = "x ** 2 = x * x";
        }
        else if (!strcmp(in->op, "**") && (isvar(in->a) != isvar(in->b))) {
            /* one operand is a variable v, the other a constant power of two */
            strcpy(v, isvar(in->a) ? in->a : in->b);
            k = power_of_two(isvar(in->a) ? in->b : in->a);
            if (k == 1) {
                strcpy(in->a, v); strcpy(in->b, v);
                strcpy(in->op, "+");
                why = "x * 2 = x + x";
            } else if (k >= 2) {
                sprintf(num, "%d", k);
            }
        }
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        strcpy(in->a, v); strcpy(in->b, num);
        strcpy(in->op, "<<");
        why = "x * 2^k = x << k";
    }
}
if (why) {
    show(in, after);
    note(3, title, before, after, why);
    changes++;
}
}
if (!changes) no_change(3, title);
return changes;
}

/* ----- 4. dead code elimination */

static int in_set(char set[][MAXN], int cnt, const char *v)
{
    int k;
    for (k = 0; k < cnt; k++) if (!strcmp(set[k], v)) return 1;
    return 0;
}

static int dead_code_elimination(void)
{
    const char *title = "Dead Code Elimination";
    static char live[MAXV][MAXN];
    int cnt = 0, i, k, changes = 0;
    char before[LINE], why[LINE];

    /* variables needed after the block: the declared list, or every result */
    if (haveLive) {
        for (k = 0; k < nLive; k++) strcpy(live[cnt++], liveOut[k]);
    } else {
        for (i = 0; i < n; i++)
            if (!prog[i].dead && !in_set(live, cnt, prog[i].dest))
                strcpy(live[cnt++], prog[i].dest);
    }

    /* walk backwards: an assignment is useful only if its result is live */
    for (i = n - 1; i >= 0; i--) {
        Instr *in = &prog[i];
        if (in->dead) continue;
        show(in, before);
        if (!in_set(live, cnt, in->dest)) {
            in->dead = 1;
            sprintf(why, "%s is never used", in->dest);
            note(4, title, before, NULL, why);
            changes++;
            continue;
        }
        /* the definition kills dest; the operands are used by this instruction */
        for (k = 0; k < cnt; k++)
            if (!strcmp(live[k], in->dest)) { strcpy(live[k], live[--cnt]); break; }
        if (isvar(in->a) && !in_set(live, cnt, in->a))
            strcpy(live[cnt++], in->a);
        if (in->op[0] && isvar(in->b) && !in_set(live, cnt, in->b))
            strcpy(live[cnt++], in->b);
    }
    if (!changes) no_change(4, title);
    return changes;
}

/* ----- input */

static char *read_operand(char *p, char *out)

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

{
    int k = 0;
    while (isspace((unsigned char)*p)) p++;
    if (*p == '-' && isdigit((unsigned char)p[1])) out[k++] = *p++;
    if (isdigit((unsigned char)*p))
        while (isdigit((unsigned char)*p) && k < MAXN - 1) out[k++] = *p++;
    else if (isalpha((unsigned char)*p) || *p == '_')
        while ((isalnum((unsigned char)*p) || *p == '_') && k < MAXN - 1) out[k++] = *p++;
    out[k] = '\0';
    return k ? p : NULL;
}

static int parse_instruction(char *p)
{
    Instr *in = &prog[n];
    memset(in, 0, sizeof *in);

    if (!(p = read_operand(p, in->dest)) || isnum(in->dest)) return 0;
    while (isspace((unsigned char)*p)) p++;
    if (*p++ != '=') return 0;
    if (!(p = read_operand(p, in->a))) return 0;
    while (isspace((unsigned char)*p)) p++;
    if (*p == '\0') return 1; /* dest = a */

    if (!strcmp(p, "***", 2) || !strcmp(p, "<<", 2) || !strcmp(p, ">>", 2)) {
        strncpy(in->op, p, 2);
        p += 2;
    } else if (strchr("+-*/%", *p)) {
        in->op[0] = *p++;
    } else {
        return 0;
    }
    if (!(p = read_operand(p, in->b))) return 0;
    while (isspace((unsigned char)*p)) p++;
    return *p == '\0'; /* dest = a op b */
}

static void parse_live(char *p)
{
    char *tok = strtok(p, ", \t");
    haveLive = 1;
    while (tok && nLive < MAXV) {
        strncpy(liveOut[nLive++], tok, MAXN - 1);
        tok = strtok(NULL, ", \t");
    }
}

static void print_program(void)
{
    int i, line = 0;
    char buf[LINE];
    for (i = 0; i < n; i++) {
        if (prog[i].dead) continue;
        show(&prog[i], buf);
        printf("    %2d: %s\n", ++line, buf);
    }
}

int main(int argc, char *argv[])
{
    FILE *fp = (argc > 1) ? fopen(argv[1], "r") : stdin;
    char text[256];
    int lineno = 0, i, changes, total, left;

    if (!fp) { perror(argv[1]); return 1; }

    while (fgets(text, sizeof text, fp)) {

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    char *s = text;
    lineno++;
    text[strcspn(text, "\r\n")] = '\0';
    while (isspace((unsigned char)*s)) s++;
    if (*s == '\0' || *s == '#') continue;
    if (!strncasecmp(s, "live:", 5)) { parse_live(s + 5); continue; }
    if (n >= MAXI || !parse_instruction(s)) {
        printf("Error: cannot parse line %d: %s\n", lineno, s);
        return 1;
    }
    n++;
}
total = n;

printf("Input Three Address Code\n");
print_program();
if (haveLive) {
    printf("Live at exit :");
    for (i = 0; i < nLive; i++) printf(" %s%s", liveOut[i], i < nLive - 1 ? ", " : "");
    printf("\n");
}

/* apply the four techniques repeatedly until nothing changes any more */
for (;;) {
    round_shown = 0;
    memset(pass_shown, 0, sizeof pass_shown);
    changes = constant_folding();
    changes += algebraic_identities();
    changes += strength_reduction();
    changes += dead_code_elimination();
    if (!changes) break;
    round_no++;
}

printf("\nOptimized Three Address Code\n");
print_program();
for (i = 0, left = 0; i < n; i++) if (!prog[i].dead) left++;
printf("\nInstructions: %d -> %d\n", total, left);
return 0;
}

```

Input (input.txt)

```

t1= 5*3
t2=a+0
t3=b*1
t4=b*d
t5=d**2
t6=b*d
live: t1, t3, t4, t5

```

The last line gives the variables needed after this block (live at exit). An assignment whose result is not in that list and is not used later is dead code.

Commands Used

```

gcc optimizer.c -o optimizer
./optimizer < input.txt

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Output:

```

● kushaal@Kushaals-MacBook-Air Assignment 7 % gcc optimizer.c -o optimizer
● kushaal@Kushaals-MacBook-Air Assignment 7 % ./optimizer < input.txt
Input Three Address Code
  1:  t1 = 5 * 3
  2:  t2 = a + 0
  3:  t3 = b * 1
  4:  t4 = b * d
  5:  t5 = d ** 2
  6:  t6 = b * d
Live at exit : t1, t3, t4, t5

[1] Constant Folding (with constant propagation)
    t1 = 5 * 3      ==>  t1 = 15      (evaluated at compile time)

[2] Algebraic Identities
    t2 = a + 0      ==>  t2 = a        (x + 0 = x)
    t3 = b * 1      ==>  t3 = b        (x * 1 = x)

[3] Strength Reduction
    t5 = d ** 2     ==>  t5 = d * d    (x ** 2 = x * x)

[4] Dead Code Elimination
    t6 = b * d      ==>  removed      (t6 is never used)
    t2 = a          ==>  removed      (t2 is never used)

Optimized Three Address Code
  1:  t1 = 15
  2:  t3 = b
  3:  t4 = b * d
  4:  t5 = d * d

Instructions: 6 -> 4

```

Result of optimizing the given three address code

t2 = a + 0 becomes t2 = a, and t6 = b * d repeats t4; since neither t2 nor t6 is needed after the block, dead code elimination removes both. The statement t5 = d ** 2 is reduced to t5 = d * d.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Test Cases:

```

kushaal@Kushaals-MacBook-Air Assignment 7 % ./optimizer < input2.txt
Input Three Address Code
1: a = 4 * 8
2: b = a + 2
3: c = x * 8
4: d = y ** 2
5: e = z * 2
6: f = w - w
7: g = c + 0
8: c = 7
9: h = 10 / 2
10: k = m * 0
Live at exit : b, c, d, e, f, g, h, k

[1] Constant Folding (with constant propagation)
a = 4 * 8      ==> a = 32      (evaluated at compile time)
b = a + 2      ==> b = 34      (constants propagated, then evaluated)
h = 10 / 2     ==> h = 5       (evaluated at compile time)

[2] Algebraic Identities
f = w - w      ==> f = 0       (x - x = 0)
g = c + 0      ==> g = c       (x + 0 = x)
k = m * 0      ==> k = 0       (x * 0 = 0)

[3] Strength Reduction
c = x * 8      ==> c = x << 3  (x * 2^k = x << k)
d = y ** 2     ==> d = y * y   (x ** 2 = x * x)
e = z * 2      ==> e = z + z   (x * 2 = x + x)

[4] Dead Code Elimination
a = 32         ==> removed     (a is never used)

Optimized Three Address Code
1: b = 34
2: c = x << 3
3: d = y * y
4: e = z + z
5: f = 0
6: g = c
7: c = 7
8: h = 5
9: k = 0

Instructions: 10 -> 9

```

Another test case: constant propagation, shifts and dead code

$a = 4 * 8$ is folded to 32 and propagated into $b = a + 2$, giving $b = 34$, after which a is dead. $c = x * 8$ becomes $c = x \ll 3$, and the earlier c is kept because g uses it before c is assigned again.

Learning Outcomes:

- Learned four local optimizations: constant folding (with constant propagation), algebraic identities, strength reduction and dead code elimination.
- Understood that dead code elimination depends on liveness: the block is scanned backwards and an assignment is removed when its result is not live.
- Learned that one optimization can enable another ($x * 1$ becomes a copy, which may then be dead), so the passes are repeated until nothing changes.
- Gained experience in parsing three address code in C and rewriting instructions safely, for example by not folding a division by zero.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 8

Date: 28-08-2026

Implementation of Code Generation

Aim:

To implement a code generator that translates three address code containing assignments, arithmetic operators, relational operators, conditional constructs and loops into 8086 style assembly code.

Tools Used:

- GCC

Code:

codegen.c

```

/*
 * Code generator : Three Address Code -> 8086 style assembly code
 *
 * Supports : assignment, arithmetic operators (+ - * / %), unary minus,
 *            relational operators (< <= > >= == !=), conditional statements
 *            (if a relop b goto L) and iterative constructs (labels + goto).
 *
 * Method : 1. read the TAC and turn it into machine level instructions
 *           (constants that must sit in a register get their own MOV)
 *           2. live variable analysis (backward data flow, repeated until it
 *           stops changing - this is what makes loops work)
 *           3. two names may share a register only if they are never live at
 *           the same time; a name that finds no free register is spilled
 *           4. print the assembly code
 *
 * Input : x = y      x = y op z      x = -y      if a relop b goto L
 *         goto L      L: (a label may also precede a statement)
 * Usage : ./codegen < input.tac      or      ./codegen input.tac
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <stdarg.h>

#ifdef NREG
#define NREG 8 /* registers R0..R7 */
#endif
#define NALLOC (NREG - 2) /* R0..R5 are allocated, R6 and R7 are */
/* scratch registers used for spilled names */

#define NAME 32
#define MAXLAB 4
#define MAXSRC 200
#define MAXNODE 500
#define MAXVAR 300

/* ----- input TAC */

enum { S_COPY, S_BIN, S_NEG, S_IF, S_GOTO, S_NOP };

typedef struct {
    int kind;
    char dest[NAME], a[NAME], b[NAME]; /* x = a op b */
    char op[3]; /* arithmetic operator, or relational */

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

char target[NAME];          /* jump target */
char lab[MAXLAB][NAME];    /* labels attached to this statement */
int nlab;
} Src;

static Src src[MAXSRC];
static int nsrc = 0;
static char pend[MAXLAB][NAME]; /* labels waiting for their statement */
static int npend = 0;

static void die(const char *msg, int lineno, const char *text)
{
    printf("Error (line %d): %s : %s\n", lineno, msg, text);
    exit(1);
}

static char *skip(char *p)
{
    while (isspace((unsigned char)*p)) p++;
    return p;
}

/* read an identifier or a number (a '-' directly before digits is part of it) */
static char *word(char *p, char *out, int minus)
{
    int k = 0;
    p = skip(p);
    if (minus && *p == '-' && isdigit((unsigned char)p[1])) out[k++] = *p++;
    if (isdigit((unsigned char)*p))
        while (isdigit((unsigned char)*p) && k < NAME - 1) out[k++] = *p++;
    else if (isalpha((unsigned char)*p) || *p == '_')
        while ((isalnum((unsigned char)*p) || *p == '_') && k < NAME - 1) out[k++] = *p++;
    out[k] = '\0';
    return k ? p : NULL;
}

static int isnum(const char *s)
{
    return isdigit((unsigned char)s[0]) || (s[0] == '-' && isdigit((unsigned char)s[1]));
}

static int keyword(char *p, const char *kw)
{
    size_t n = strlen(kw);
    return !strncmp(p, kw, n) && !isalnum((unsigned char)p[n]) && p[n] != '_';
}

static void parse_line(char *p, int lineno)
{
    Src *s = &src[nsrc];
    char *q, w[NAME];
    const char *whole = p;

    memset(s, 0, sizeof *s);
    for (;;) {
        q = word(p, w, 0);
        if (q && !isdigit((unsigned char)w[0]) && *skip(q) == ':' && !keyword(w, "if")) {
            if (npend >= MAXLAB) die("too many labels", lineno, whole);
            strcpy(pend[npend++], w);
            p = skip(q) + 1;
        } else break;
    }
    p = skip(p);
    if (!*p) return;

    if (keyword(p, "goto")) {
        s->kind = S_GOTO;
    }

```


Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (!(q = word(p + 4, s->target, 0)) || *skip(q)) die("bad goto", lineno, whole);
} else if (keyword(p, "if")) {
    s->kind = S_IF;
    if (!(p = word(p + 2, s->a, 1))) die("bad if", lineno, whole);
    p = skip(p);
    if (!strcmp(p, "<=", 2) || !strcmp(p, ">=", 2) ||
        !strcmp(p, "==", 2) || !strcmp(p, "!=", 2)) {
        strcpy(s->op, p, 2); p += 2;
    } else if (*p == '<' || *p == '>') {
        s->op[0] = *p++;
    } else die("relational operator expected", lineno, whole);
    if (!(p = word(p, s->b, 1))) die("bad if", lineno, whole);
    p = skip(p);
    if (!keyword(p, "goto")) die("goto expected", lineno, whole);
    if (!(q = word(p + 4, s->target, 0)) || *skip(q)) die("bad goto", lineno, whole);
} else {
    if (!(p = word(p, s->dest, 0)) || !isnum(s->dest)) die("bad statement", lineno, whole);
    p = skip(p);
    if (*p != '=' || p[1] == '=') die("'=' expected", lineno, whole);
    p = skip(p + 1);
    if (*p == '-' && !isdigit((unsigned char)p[1])) { /* x = -y */
        s->kind = S_NEG;
        if (!(p = word(p + 1, s->a, 0))) die("bad operand", lineno, whole);
    } else {
        if (!(p = word(p, s->a, 1))) die("bad operand", lineno, whole);
        p = skip(p);
        if (!*p) s->kind = S_COPY;
        else if (strchr("+-*/%", *p)) {
            s->kind = S_BIN;
            s->op[0] = *p++;
            if (!(p = word(p, s->b, 1))) die("bad operand", lineno, whole);
        } else die("operator expected", lineno, whole);
    }
    if (*skip(p)) die("unexpected text", lineno, whole);
}
memcpy(s->lab, pend, sizeof pend);
s->nlab = npend;
npend = 0;
nsrcc++;
}

/* ----- machine level code */

enum { N_MOVI, N_COPY, N_BIN, N_NEG, N_CMPJ, N_JMP, N_NOP };

typedef struct {
    int      kind;
    int      d, a, b;          /* variable numbers, -1 if unused */
    long long imm;            /* N_MOVI : constant loaded */
    char      op[3];           /* arithmetic or relational operator */
    char      tlabel[NAME];    /* N_CMPJ / N_JMP : target label */
    int      target;           /* ... and the node that carries it */
    char      lab[MAXLAB][NAME]; /* labels on this node */
    int      nlab;
} Node;

static Node node[MAXNODE];
static int  nn = 0;

static char names[MAXVAR][NAME]; /* variable table */
static int  isconst[MAXVAR];     /* 1 : register holding a constant */
static long long cval[MAXVAR];
static int  nvar = 0;

static int var(const char *name)
{
    int v;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    for (v = 0; v < nvar; v++)
        if (!isconst[v] && !strcmp(names[v], name)) return v;
    if (nvar >= MAXVAR) { printf("Error: too many variables\n"); exit(1); }
    strcpy(names[nvar], name);
    isconst[nvar] = 0;
    return nvar++;
}

static int newconst(long long value)
{
    if (nvar >= MAXVAR) { printf("Error: too many variables\n"); exit(1); }
    sprintf(names[nvar], "%lld", value);
    isconst[nvar] = 1;
    cval[nvar] = value;
    return nvar++;
}

static Node *add(int kind, int d, int a, int b)
{
    Node *n;
    if (nn >= MAXNODE) { printf("Error: program too long\n"); exit(1); }
    n = &node[nn++];
    memset(n, 0, sizeof *n);
    n->kind = kind; n->d = d; n->a = a; n->b = b;
    return n;
}

/* Operand of a statement as a variable number; a constant is first loaded into
   a register of its own (this MOV is emitted before the statement itself). */
static int operand(const char *s)
{
    int v;
    if (!isnum(s)) return var(s);
    v = newconst(atoll(s));
    add(N_MOVI, v, -1, -1)->imm = cval[v];
    return v;
}

static void attach(int at, const Src *s)
{
    memcpy(node[at].lab, s->lab, sizeof s->lab);
    node[at].nlab = s->nlab;
}

/* May the constants of the conditional jump 'i' be loaded before its label?
   Yes if every jump to that label comes from below (a loop back edge), because
   then the label is only reached again after the constant was loaded. */
static int hoistable(int i)
{
    int j, k;
    if (src[i].kind != S_IF || src[i].nlab == 0) return 0;
    for (j = 0; j < i; j++) {
        if (src[j].kind != S_IF && src[j].kind != S_GOTO) continue;
        for (k = 0; k < src[i].nlab; k++)
            if (!strcmp(src[j].target, src[i].lab[k])) return 0;
    }
    return 1;
}

static void build(int hoist)
{
    int i, first, a, b, d, at;
    nn = 0; nvar = 0;
    for (i = 0; i < nsrc; i++) {
        Src *s = &src[i];
        first = nn;
        switch (s->kind) {

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    case S_COPY:
        d = var(s->dest);
        if (isnum(s->a)) add(N_MOVI, d, -1, -1->imm = atoll(s->a);
        else add(N_COPY, d, var(s->a), -1);
        at = first;
        break;
    case S_NEG:
        d = var(s->dest);
        if (isnum(s->a)) add(N_MOVI, d, -1, -1->imm = -atoll(s->a);
        else add(N_NEG, d, var(s->a), -1);
        at = first;
        break;
    case S_BIN:
        a = operand(s->a); b = operand(s->b); d = var(s->dest);
        strcpy(add(N_BIN, d, a, b)->op, s->op);
        at = first;
        break;
    case S_IF:
        a = operand(s->a); b = operand(s->b);
        strcpy(add(N_CMPJ, -1, a, b)->op, s->op);
        strcpy(node[nn - 1].tlabel, s->target);
        at = (hoist && hoistable(i)) ? nn - 1 : first;
        break;
    case S_GOTO:
        strcpy(add(N_JMP, -1, -1, -1->tlabel, s->target);
        at = first;
        break;
    default:
        add(N_NOP, -1, -1, -1);
        at = first;
    }
    attach(at, s);
}
/* resolve the jump targets */
for (i = 0; i < nn; i++) {
    int k, m, found = 0;
    if (node[i].kind != N_CMPJ && node[i].kind != N_JMP) continue;
    for (k = 0; k < nn && !found; k++)
        for (m = 0; m < node[k].nlab; m++)
            if (!strcmp(node[k].lab[m], node[i].tlabel)) {
                node[i].target = k; found = 1; break;
            }
    if (!found) { printf("Error: label %s is not defined\n", node[i].tlabel); exit(1); }
}

/* ----- live variable analysis */

static char in[MAXNODE][MAXVAR], out[MAXNODE][MAXVAR]; /* live before / after a node */
static char exitLive[MAXVAR];

static int is_temp(const char *s) /* t1, t2 ... */
{
    int k;
    if (s[0] != 't' || !s[1]) return 0;
    for (k = 1; s[k]; k++) if (!isdigit((unsigned char)s[k])) return 0;
    return 1;
}

static void liveness(void)
{
    int changed = 1, k, v;
    memset(in, 0, sizeof in); memset(out, 0, sizeof out); memset(exitLive, 0, sizeof exitLive);
    for (v = 0; v < nvar; v++) /* results of the program stay alive */
        if (!isconst[v] && !is_temp(names[v])) exitLive[v] = 1;

    while (changed) {

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

changed = 0;
for (k = nn - 1; k >= 0; k--) {
    Node *n = &node[k];
    char o[MAXVAR], i2[MAXVAR];
    memset(o, 0, sizeof o);
    if (n->kind != N_JMP) { /* fall through to the next node */
        if (k + 1 < nn) for (v = 0; v < nvar; v++) o[v] |= in[k + 1][v];
        else for (v = 0; v < nvar; v++) o[v] |= exitLive[v];
    }
    if (n->kind == N_CMPJ || n->kind == N_JMP) /* jump to the target */
        for (v = 0; v < nvar; v++) o[v] |= in[n->target][v];
    memcpy(i2, o, sizeof i2);
    if (n->d >= 0) i2[n->d] = 0; /* in = use + (out - def) */
    if (n->a >= 0) i2[n->a] = 1;
    if (n->b >= 0) i2[n->b] = 1;
    if (memcmp(o, out[k], sizeof o) || memcmp(i2, in[k], sizeof i2)) {
        memcpy(out[k], o, sizeof o);
        memcpy(in[k], i2, sizeof i2);
        changed = 1;
    }
}
}

/* ----- register allocation */

static char interf[MAXVAR][MAXVAR];
static int reg[MAXVAR]; /* register number, -1 = not allocated */
static int spilled[MAXVAR]; /* 1 = lives in memory */

static void interfere(int x, int y)
{
    if (x != y) interf[x][y] = interf[y][x] = 1;
}

static void allocate(void)
{
    int order[MAXVAR], seen[MAXVAR], cnt = 0, k, v, w, r, c;
    memset(interf, 0, sizeof interf); memset(seen, 0, sizeof seen);
    for (v = 0; v < nvar; v++) { reg[v] = -1; spilled[v] = 0; }

    /* a name that is defined interferes with everything alive right after it
       (except the source of a copy, which holds the same value) */
    for (k = 0; k < nn; k++) {
        Node *n = &node[k];
        if (n->d < 0) continue;
        for (w = 0; w < nvar; w++)
            if (out[k][w] && !(n->kind == N_COPY && w == n->a)) interfere(n->d, w);
    }
    /* d = a - b : the result register must not be the register of b, otherwise
       b would be overwritten before it is used (+ and * can simply be swapped) */
    for (k = 0; k < nn; k++)
        if (node[k].kind == N_BIN && node[k].op[0] != '+' && node[k].op[0] != '*')
            interfere(node[k].d, node[k].b);
    /* names that are alive on entry (used before being assigned) all coexist */
    for (v = 0; v < nvar; v++)
        for (w = v + 1; w < nvar; w++)
            if (in[0][v] && in[0][w]) interfere(v, w);

    /* colour the names in the order in which they first appear */
    for (k = 0; k < nn; k++) {
        int list[3] = { node[k].a, node[k].b, node[k].d };
        for (c = 0; c < 3; c++)
            if (list[c] >= 0 && !seen[list[c]]) { seen[list[c]] = 1; order[cnt++] = list[c]; }
    }
    for (k = 0; k < cnt; k++) {
        int used[NALLOC + 1], pick = -1;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    v = order[k];
    memset(used, 0, sizeof used);
    for (w = 0; w < nvar; w++)
        if (interf[v][w] && reg[w] >= 0) used[reg[w]] = 1;
    /* x = y, x = -y, x = y op z : x likes to reuse the register of y */
    for (c = 0; c < nn && pick < 0; c++) {
        int p = -1;
        if (node[c].kind != N_COPY && node[c].kind != N_NEG && node[c].kind != N_BIN)
            continue;
        if (node[c].d == v) p = node[c].a; else if (node[c].a == v) p = node[c].d;
        if (p >= 0 && reg[p] >= 0 && !used[reg[p]]) pick = reg[p];
    }
    for (r = 0; r < NALLOC && pick < 0; r++)
        if (!used[r]) pick = r;
    if (pick >= 0) reg[v] = pick; else spilled[v] = 1;
}
}

/* ----- code emission */

#define S1 (NREG - 2)          /* scratch registers          */
#define S2 (NREG - 1)

static char pendlab[MAXLAB][NAME];
static int npendlab = 0;

static void emit(const char *fmt, ...)
{
    va_list ap;
    int k;
    for (k = 0; k < npendlab - 1; k++) printf("%s:\n", pendlab[k]);
    if (npendlab) printf("%s: ", pendlab[npendlab - 1]);
    npendlab = 0;
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    printf("\n");
}

static const char *opnd(int v)          /* how an operand is written */
{
    static char buf[4][NAME + 4];
    static int turn = 0;
    char *b = buf[turn++ & 3];
    if (spilled[v]) {
        if (isconst[v]) sprintf(b, "%lld", cval[v]);
        else             sprintf(b, "[%s]", names[v]);
    } else sprintf(b, "R%d", reg[v]);
    return b;
}

static const char *mnemonic(const char *op)
{
    switch (op[0]) {
        case '+': return "ADD";
        case '-': return "SUB";
        case '*': return "MUL";
        case '/': return "DIV";
        default : return "MOD";
    }
}

static const char *jump(const char *rel)
{
    if (!strcmp(rel, "<")) return "JL";
    if (!strcmp(rel, "<=")) return "JLE";
    if (!strcmp(rel, ">")) return "JG";
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (!strcmp(rel, ">=")) return "JGE";
    if (!strcmp(rel, "==")) return "JE";
    return "JNE";
}

static void generate(void)
{
    int k, m;
    for (k = 0; k < nn; k++) {
        Node *n = &node[k];
        npendlab = 0;
        for (m = 0; m < n->nlab; m++) strcpy(pendlab[npendlab++], n->lab[m]);

        switch (n->kind) {
            case N_MOVI:
                if (isconst[n->d] && spilled[n->d]) break; /* used as an immediate */
                if (spilled[n->d]) emit("MOV %s, #lld", opnd(n->d), n->imm);
                else emit("MOV R%d, #lld", reg[n->d], n->imm);
                break;
            case N_COPY:
                if (!spilled[n->d] && !spilled[n->a] && reg[n->d] == reg[n->a]) break;
                if (spilled[n->d] && spilled[n->a]) {
                    emit("MOV R%d, %s", S1, opnd(n->a));
                    emit("MOV %s, R%d", opnd(n->d), S1);
                } else emit("MOV %s, %s", opnd(n->d), opnd(n->a));
                break;
            case N_NEG: {
                int r = spilled[n->d] ? S1 : reg[n->d];
                if (spilled[n->a] || reg[n->a] != r) emit("MOV R%d, %s", r, opnd(n->a));
                emit("NEG R%d", r);
                if (spilled[n->d]) emit("MOV %s, R%d", opnd(n->d), r);
                break;
            }
            case N_BIN: {
                int r = spilled[n->d] ? S1 : reg[n->d];
                int a_here = !spilled[n->a] && reg[n->a] == r;
                int b_here = !spilled[n->b] && reg[n->b] == r;
                int comm = (n->op[0] == '+' || n->op[0] == '*');
                if (b_here && !a_here && comm) { /* r already holds b */
                    emit("%s R%d, %s", mnemonic(n->op), r, opnd(n->a));
                } else if (b_here && !a_here) { /* x = y - x : r holds b */
                    emit("MOV R%d, %s", S1, opnd(n->a));
                    emit("%s R%d, R%d", mnemonic(n->op), S1, r);
                    emit("MOV R%d, R%d", r, S1);
                } else {
                    if (!a_here) emit("MOV R%d, %s", r, opnd(n->a));
                    emit("%s R%d, %s", mnemonic(n->op), r, opnd(n->b));
                }
                if (spilled[n->d]) emit("MOV %s, R%d", opnd(n->d), r);
                break;
            }
            case N_CMPJ:
                if (spilled[n->a]) { /* CMP needs a register first */
                    emit("MOV R%d, %s", S1, opnd(n->a));
                    emit("CMP R%d, %s", S1, opnd(n->b));
                } else emit("CMP %s, %s", opnd(n->a), opnd(n->b));
                emit("%s %s", jump(n->op), n->tlabel);
                break;
            case N_JMP:
                emit("JMP %s", n->tlabel);
                break;
            default:
                break;
        }
    }
    for (m = 0; m < npendlab; m++) /* labels of nodes that produced no code */
        printf("%s:\n", pendlab[m]);
    npendlab = 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    }
}

/* ----- main */

static void print_tac(void)
{
    int i, m;
    for (i = 0; i < nsrc; i++) {
        Src *s = &src[i];
        for (m = 0; m < s->nlab; m++)
            printf(s->kind == S_NOP && m == s->nlab - 1 ? "%s:" : "%s: ", s->lab[m]);
        switch (s->kind) {
            case S_COPY: printf("%s = %s", s->dest, s->a); break;
            case S_NEG:  printf("%s = -%s", s->dest, s->a); break;
            case S_BIN:  printf("%s = %s %s %s", s->dest, s->a, s->op, s->b); break;
            case S_IF:   printf("if %s %s %s goto %s", s->a, s->op, s->b, s->target); break;
            case S_GOTO: printf("goto %s", s->target); break;
            default: break;
        }
        printf("\n");
    }
}

int main(int argc, char *argv[])
{
    FILE *fp = (argc > 1) ? fopen(argv[1], "r") : stdin;
    char text[256];
    int lineno = 0, v, hoist, k, any;

    if (!fp) { perror(argv[1]); return 1; }
    while (fgets(text, sizeof text, fp)) {
        char *s = text, *c;
        lineno++;
        if ((c = strstr(s, "//")) != NULL) *c = '\0';
        if ((c = strchr(s, '#')) != NULL) *c = '\0';
        s[strcspn(s, "\r\n")] = '\0';
        s = skip(s);
        if (!*s) continue;
        if (nsrc >= MAXSRC - 1) die("program too long", lineno, s);
        parse_line(s, lineno);
    }
    if (npend) { /* labels at the very end */
        memset(&src[nsrc], 0, sizeof(Src));
        src[nsrc].kind = S_NOP;
        memcpy(src[nsrc].lab, pend, sizeof pend);
        src[nsrc].nlab = npend;
        nsrc++;
    }

    /* load loop constants before the loop if that is safe, otherwise inside */
    for (hoist = 1; ; hoist = 0) {
        build(hoist);
        liveness();
        for (v = 0, any = 0; v < nvar; v++) if (isconst[v] && in[0][v]) any = 1;
        if (!hoist || !any) break;
    }
    allocate();

    printf("Three Address Code\n");
    print_tac();
    printf("\nAssembly Code\n");
    generate();

    printf("\nRegister Allocation\n");
    for (v = 0; v < nvar; v++) {
        if (isconst[v]) continue;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    printf("    %-6s -> ", names[v]);
    if (spilled[v]) printf("memory [%s]\n", names[v]);
    else             printf("R%d\n", reg[v]);
}
for (v = 0; v < nvar; v++) {
    if (!isconst[v]) continue;
    for (k = 0, any = 0; k < nn; k++) if (node[k].kind == N_MOVI && node[k].d == v) any = 1;
    if (any && !spilled[v]) printf("    %-6s -> R%d\n", names[v], reg[v]);
}
for (v = 0; v < nvar; v++)
    if (!isconst[v] && in[0][v])
        printf("Note: %s is used before it is assigned (treated as an input)\n", names[v]);
return 0;
}

```

Input (input.txt) - three address code of $x=0$; for($i=1$; $i \leq 10$; $i++$) $x=x+1$

```

x=0
i=1
L3: if i<=10 goto L1
goto L2
L1: t1=x+i
x=t1
t2=i+1
i=t2
goto L3
L2:

```

Commands Used

```

gcc codegen.c -o codegen
./codegen < input.txt

```


Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Output:

```

#1      -> R3
● kushaal@Kushaals-MacBook-Air Assignment 8 % gcc codegen.c -o codegen
● kushaal@Kushaals-MacBook-Air Assignment 8 % cat input.txt
x=0
i=1
L3: if i<=10 goto L1
goto L2
L1: t1=x+i
x=t1
t2=i+1
i=t2
goto L3
L2:
● kushaal@Kushaals-MacBook-Air Assignment 8 % ./codegen < input.txt
Three Address Code
x = 0
i = 1
L3: if i <= 10 goto L1
goto L2
L1: t1 = x + i
x = t1
t2 = i + 1
i = t2
goto L3
L2:

Assembly Code
MOV R0, #0
MOV R1, #1
MOV R2, #10
L3: CMP R1, R2
JLE L1
JMP L2
L1: ADD R0, R1
MOV R3, #1
ADD R1, R3
JMP L3
L2:

Register Allocation
x      -> R0
i      -> R1
t1     -> R0
t2     -> R1
#10    -> R2
#1     -> R3

```

Assembly code produced for the given three address code

x = t1 and i = t2 need no instruction, because t1 and t2 are placed in the registers of x and i, which are no longer needed. The constant 10 does not change inside the loop, so it is loaded into R2 once, before the label L3.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Test Cases:

```

#1 -> R3
• kushaal@Kushaals-MacBook-Air Assignment 8 % cat input2.txt
a = 8
b = 3
if a > b goto L1
c = a - b
goto L2
L1: c = a * b
L2: t1 = c % 5
d = -t1
• kushaal@Kushaals-MacBook-Air Assignment 8 % ./codegen < input2.txt
Three Address Code
a = 8
b = 3
if a > b goto L1
c = a - b
goto L2
L1: c = a * b
L2: t1 = c % 5
d = -t1

Assembly Code
MOV R0, #8
MOV R1, #3
CMP R0, R1
JG L1
MOV R2, R0
SUB R2, R1
JMP L2
L1: MOV R2, R0
MUL R2, R1
L2: MOV R3, #5
MOV R4, R2
MOD R4, R3
NEG R4

Register Allocation
a    -> R0
b    -> R1
c    -> R2
t1   -> R4
d    -> R4
#5   -> R3

```

Another test case: conditional statement with arithmetic operators

Learning Outcomes:

- Learned how to translate three address code into target instructions (MOV, ADD, SUB, MUL, DIV, CMP and the jumps JL, JLE, JG, JGE, JE, JNE).
- Understood live variable analysis for programs with loops: it is a backward data flow problem that is repeated until nothing changes.
- Learned register allocation: two names can share a register only if they are never live together, and a name with no free register is kept in memory.
- Learned to translate relational operators and loops into a compare followed by a conditional jump, and to load loop-invariant constants before the loop only when that is safe.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 9

Date: 01-10-2026

Implementation of Mini Compiler using Lex and Yacc

Aim:

To implement a mini compiler with Lex and Yacc that takes a C++ source program and passes it through lexical analysis, syntax analysis, intermediate code generation, code optimization and code generation, producing 8086-style assembly code.

Tools Used:

- Lex (Flex) - recognizes keywords, identifiers, constants, operators and special characters as tokens, and reports lexical errors.
- Yacc (Bison) - defines the grammar, checks the syntax and declarations, and generates three address code through syntax directed translation rules.
- C Programming Language - used to write the symbol table, the code optimizer and the code generator.
- C Compiler (GCC/CC) - compiles the C code generated by Lex and Yacc together with the other C files.
- Linux Terminal (Ubuntu on WSL) - used to run the Lex, Yacc and gcc commands.
- C++ Input File (input.cpp) - the source program given to the compiler as input.

Code:

scanner.l

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include "y.tab.h"

/* Phase 1 : every token is listed together with its class */
static void show(const char *kind)
{
    printf("%s - %s\n", yytext, kind);
}

static void lexError(const char *msg)
{
    printf("Lexical error at line %d: %s\n", yylineno, msg);
    exit(1);
}
}%

%option noyywrap nounput noinput yylineno
%x COMMENT

DIGIT    [0-9]
LETTER   [a-zA-Z_]

%%

"//".*      { /* single line comment */ }
```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

"/*"          { BEGIN(COMMENT); }
<COMMENT>"*/" { BEGIN(INITIAL); }
<COMMENT>\n    { }
<COMMENT>.      { }
<COMMENT><<EOF>> { lexError("unterminated comment"); }

^[ \t]*"#"[^\\n]* { printf("%s - preprocessor directive\n",
                          yytext + strspn(yytext, " \t")); }

"using"        { show("keyword"); return USING; }
"namespace"    { show("keyword"); return NAMESPACE; }
"int"          { show("keyword"); return INT; }
"void"         { show("keyword"); return VOID; }
"return"       { show("keyword"); return RETURN; }
"if"           { show("keyword"); return IF; }
"else"         { show("keyword"); return ELSE; }
"while"        { show("keyword"); return WHILE; }
"main"         { show("identifier"); return MAIN; }
"std::"?"cout" { show("output stream (function call)"); return COUT; }
"std::"?"endl" { show("stream manipulator"); return ENDL; }
"std::"?"pow"   { show("function call"); return POWFN; }

{LETTER}{(LETTER)|(DIGIT)}* { show("identifier");
                              yylval.str = strdup(yytext); return ID; }

{DIGIT}+          { if (strlen(yytext) > 10 || atoll(yytext) > INT_MAX)
                    lexError("integer constant too large");
                    show("integer constant");
                    yylval.str = strdup(yytext); return NUM; }

\"([^\\"\\n]|\\.)*\" { show("string constant");
                    yylval.str = strdup(yytext); return STR; }
\"([^\\"\\n]|\\.)*   { lexError("unterminated string"); }

"<<"          { show("stream insertion operator"); return SHL; }
"<="|">="|"=="|"!="|"<|">" { show("relational operator");
                              yylval.str = strdup(yytext); return RELOP; }
"="            { show("assignment operator"); return '='; }
["+/*%/]      { show("arithmetic operator"); return yytext[0]; }
"([{}\\[\\];,]" { show("special character"); return yytext[0]; }

[ \t\r\n]+     { /* skip white space */ }
.              { char m[40];
                sprintf(m, "unknown character '%s'", yytext);
                lexError(m); }

%%

```

parser.y

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "compiler.h"

int yylex(void);
void yyerror(const char *s);
extern int  yylineno;
extern char *yytext;
extern FILE *yyin;

static int phase2Started = 0;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

static void startPhase2(void)
{
    if (!phase2Started) printf("\n----- PHASE 2: SYNTAX ANALYSIS ----- \n");
    phase2Started = 1;
}

/* ----- symbol table */

#define MAXSYM 200

typedef struct {
    char name[NAMELEN];
    char type[12];
    int bytes, addr;
    char value[NAMELEN]; /* initial value if it is a constant, else "-" */
    int depth;           /* block in which the variable is declared */
    int hidden;          /* the block has ended: no longer visible */
    int assigned;        /* a value has been given to the variable */
} Sym;

static Sym sym[MAXSYM];
static int nsym = 0, nextAddr = 1000, depth = 0;

static int findSym(const char *name)
{
    int k;
    for (k = nsym - 1; k >= 0; k--)
        if (!sym[k].hidden && !strcmp(sym[k].name, name)) return k;
    return -1;
}

static void semError(const char *msg, const char *name)
{
    startPhase2();
    printf("Semantic error at line %d: %s '%s'\n", yylineno, msg, name);
    exit(1);
}

static void addSym(const char *name, const char *value, int assigned)
{
    if (findSym(name) >= 0) semError("variable already declared:", name);
    if (isTemp(name)) semError("name is reserved for temporaries:", name);
    if (nsym >= MAXSYM) semError("too many variables:", name);
    strcpy(sym[nsym].name, name);
    strcpy(sym[nsym].type, "int");
    sym[nsym].bytes = 4;
    sym[nsym].addr = nextAddr;
    nextAddr += 4;
    strcpy(sym[nsym].value, value);
    sym[nsym].depth = depth;
    sym[nsym].hidden = 0;
    sym[nsym].assigned = assigned;
    nsym++;
}

static void endBlock(void)
{
    int k;
    for (k = 0; k < nsym; k++)
        if (sym[k].depth == depth) sym[k].hidden = 1;
    depth--;
}

static void checkDeclared(const char *name)
{
    if (findSym(name) < 0) semError("undeclared variable", name);
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

static void checkUse(const char *name)
{
    int k = findSym(name);
    if (k < 0)          semError("undeclared variable", name);
    if (!sym[k].assigned) semError("variable might not have been initialized:", name);
}

static void markAssigned(const char *name)
{
    sym[findSym(name)].assigned = 1;
}

/* ----- code generation helpers */

static char *binary(const char *op, const char *a, const char *b)
{
    char *t = newTemp();
    emit(op, a, b, t);
    return t;
}

static const char *negate(const char *rel)          /* jump when the condition is false */
{
    if (!strcmp(rel, "<")) return ">=";
    if (!strcmp(rel, ">")) return "<=";
    if (!strcmp(rel, "<=")) return ">";
    if (!strcmp(rel, ">=")) return "<";
    if (!strcmp(rel, "==")) return "!=";
    return "=="
}
%}

%union { char *str; }

%token <str> ID NUM STR RELOP
%token USING NAMESPACE VOID MAIN INT RETURN IF ELSE WHILE COUT ENDL POWFN SHL
%type <str> expr cond

%left '+' '-'
%left '*' '/' '%'
%right UMINUS

%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE

%%

program
: usings INT MAIN '(' params ')' mainblock
;

/* using namespace std;  (#include lines are removed by the scanner) */
usings
: usings USING NAMESPACE ID ';'
| /* empty */
;

params
: VOID
| /* empty */
;

/* body of main : statements, then an optional return expr; */
mainblock
: '{'
    stmts opt_return '}'
    { depth++; }
    { endBlock(); }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

;

opt_return
: RETURN expr ';'
| /* empty */
;

block
: '{'                                { depth++; }
  stmts '}'                          { endBlock(); }
;

stmts
: stmts stmt
| /* empty */
;

stmt
: decl ';'
| assign
| ifstmt
| whilestmt
| print
| block
;

/* Declaration statement : int a, b = 5; */
decl
: INT declist
;

declist
: declitem
| declist ',' declitem
;

declitem
: ID                                { addSym($1, "-", 0); }
| ID '=' expr                      { addSym($1, isNum($3) ? $3 : "-", 1);
                                   emit("=", $3, "", $1); }
;

/* Assignment statement : a = b + 1; */
assign
: ID '=' expr ';'                  { checkDeclared($1);
                                   emit("=", $3, "", $1);
                                   markAssigned($1); }
;

/* Conditional statement : if (c) s | if (c) s else s */
ifstmt
: IF '(' cond ')' stmt %prec LOWER_THAN_ELSE
  { emit("label", "", "", $3); }
| IF '(' cond ')' stmt ELSE { char *end = newLabel();
  emit("goto", "", "", end);
  emit("label", "", "", $3);
  $<str>$ = end; }
  stmt                            { emit("label", "", "", $<str>7); }
;

/* Looping statement : while (c) s */
whilestmt
: WHILE                            { char *top = newLabel();
  emit("label", "", "", top);
  $<str>$ = top; }
  '(' cond ')' stmt               { emit("goto", "", "", $<str>2);
  emit("label", "", "", $4); }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

;

/* Output statement : cout << "text" << a + b << endl; */
print
: COUT outlist '; '
;

outlist
: SHL outitem
| outlist SHL outitem
;

outitem
: STR { emit("print", $1, "", ""); }
| ENDL { emit("print", "\\n\\n\\n", "", ""); }
| expr { emit("print", $1, "", ""); }
;

/* the generated jump leaves the statement when the condition is FALSE */
cond
: expr RELOP expr { char *out = newLabel();
emit(negate($2), $1, $3, out);
$$ = out; }
;

expr
: expr '+' expr { $$ = binary("+", $1, $3); }
| expr '-' expr { $$ = binary("-", $1, $3); }
| expr '*' expr { $$ = binary("*", $1, $3); }
| expr '/' expr { $$ = binary("/", $1, $3); }
| expr '%' expr { $$ = binary("%", $1, $3); }
| POWFN '(' expr ',' expr ')' { $$ = binary("^", $3, $5); }
| '-' expr %prec UMINUS { char *t = newTemp();
emit("neg", $2, "", t);
$$ = t; }
| '(' expr ')' { $$ = $2; }
| NUM { $$ = $1; }
| ID { checkUse($1); $$ = $1; }
;

%%

void yyerror(const char *s)
{
startPhase2();
printf("Syntactically incorrect: %s near '%s' at line %d\\n", s, yytext, yylineno);
exit(1);
}

int main(int argc, char *argv[])
{
const char *path = (argc > 1) ? argv[1] : "input.cpp";
FILE *asmFile;
int k, before;

yyin = fopen(path, "r");
if (!yyin) {
printf("Cannot open %s\\n", path);
return 1;
}

printf("----- PHASE 1: LEXICAL ANALYSIS -----\\n");
yyparse();
startPhase2();
printf("Syntactically correct\\n");

```


Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

printf("\n----- SYMBOL TABLE -----\n");
printf("%-12s %-8s %-8s %-10s %-10s\n", "Name", "Type", "Bytes", "Address", "Value");
for (k = 0; k < nsym; k++)
    printf("%-12s %-8s %-8d %-10d %-10s\n",
           sym[k].name, sym[k].type, sym[k].bytes, sym[k].addr, sym[k].value);

printf("\n----- PHASE 3: INTERMEDIATE CODE (TAC) -----\n");
printTAC();
before = liveCount();

printf("\n----- PHASE 4: OPTIMIZED CODE -----\n");
optimize();
printTAC();
printf("\n");
printOptimizationReport();
printf("Instructions: %d -> %d\n", before, liveCount());

printf("\n----- PHASE 5: TARGET CODE (8086 style) -----\n");
asmFile = fopen("output.asm", "w");
if (!asmFile) {
    printf("Cannot create output.asm\n");
    return 1;
}
generate(asmFile);
fclose(asmFile);
printf("\nAssembly code written to output.asm\n");
return 0;
}

```

compiler.h

```

/*
 * Mini compiler - shared declarations
 *
 * Intermediate code is kept as a table of quadruples (op, arg1, arg2, result):
 *
 * + - * / % ^      result = arg1 op arg2
 * neg             result = -arg1
 * =               result = arg1
 * < <= > >= == !=   if arg1 op arg2 goto result
 * goto            goto result
 * label           result :
 * print           print arg1    (cout << arg1)
 */
#ifndef COMPILER_H
#define COMPILER_H

#include <stdio.h>

#define MAXQ 1000 /* maximum number of quadruples */
#define NAMELEN 128 /* maximum length of a name or string */

typedef struct {
    char op[8];
    char a1[NAMELEN], a2[NAMELEN], res[NAMELEN];
    int del; /* 1 = removed by the optimizer */
} Quad;

extern Quad q[MAXQ];
extern int nq;

/* quad.c */
void emit(const char *op, const char *a1, const char *a2, const char *res);
char *newTemp(void);
char *newLabel(void);

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

int isTemp(const char *s);          /* t1, t2, ...          */
int isNum(const char *s);          /* integer constant    */
int isStr(const char *s);          /* "string constant"   */
int isVar(const char *s);          /* name of a source variable */
int isBin(const char *op);         /* + - * / % ^         */
int isRel(const char *op);         /* < <= > >= == !=      */
int defines(const Quad *x);        /* quad assigns a value to x->res */
void printTAC(void);
int liveCount(void);

/* optimizer.c */
void optimize(void);
void printOptimizationReport(void);

/* codegen.c */
void generate(FILE *asmFile);

#endif

```

quad.c

```

/* Quadruple table : storage, temporaries, labels and printing */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "compiler.h"

Quad q[MAXQ];
int nq = 0;

static int tcount = 0, lcount = 0;

void emit(const char *op, const char *a1, const char *a2, const char *res)
{
    if (nq >= MAXQ) {
        printf("Error: program is too long\n");
        exit(1);
    }
    strcpy(q[nq].op, op);
    strcpy(q[nq].a1, a1);
    strcpy(q[nq].a2, a2);
    strcpy(q[nq].res, res);
    q[nq].del = 0;
    nq++;
}

char *newTemp(void)
{
    char buf[16];
    sprintf(buf, "t%d", ++tcount);
    return strdup(buf);
}

char *newLabel(void)
{
    char buf[16];
    sprintf(buf, "L%d", ++lcount);
    return strdup(buf);
}

int isTemp(const char *s)
{
    int k;
    if (s[0] != 't' || !s[1]) return 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    for (k = 1; s[k]; k++)
        if (!isdigit((unsigned char)s[k])) return 0;
    return 1;
}

int isNum(const char *s)
{
    if (*s == '-') s++;
    if (!*s) return 0;
    while (*s)
        if (!isdigit((unsigned char)*s++)) return 0;
    return 1;
}

int isStr(const char *s)
{
    return s[0] == '\'';
}

int isVar(const char *s)
{
    return (isalpha((unsigned char)s[0]) || s[0] == '_') && !isTemp(s);
}

int isBin(const char *op)
{
    return op[0] && !op[1] && strchr("+-*/%^", op[0]);
}

int isRel(const char *op)
{
    return !strcmp(op, "<") || !strcmp(op, ">") || !strcmp(op, "<=") ||
        !strcmp(op, ">=") || !strcmp(op, "==") || !strcmp(op, "!=");
}

int defines(const Quad *x)
{
    return isBin(x->op) || !strcmp(x->op, "neg") || !strcmp(x->op, "=");
}

int liveCount(void)
{
    int k, n = 0;
    for (k = 0; k < nq; k++)
        if (!q[k].del) n++;
    return n;
}

void printTAC(void)
{
    int k;
    for (k = 0; k < nq; k++) {
        const Quad *x = &q[k];
        if (x->del) continue;
        if (!strcmp(x->op, "label"))
            printf("%s:\n", x->res);
        else if (!strcmp(x->op, "goto"))
            printf("    goto %s\n", x->res);
        else if (isRel(x->op))
            printf("    if %s %s %s goto %s\n", x->a1, x->op, x->a2, x->res);
        else if (!strcmp(x->op, "print"))
            printf("    print %s\n", x->a1);
        else if (!strcmp(x->op, "="))
            printf("    %s = %s\n", x->res, x->a1);
        else if (!strcmp(x->op, "neg"))
            printf("    %s = -%s\n", x->res, x->a1);
        else

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        printf("    %s = %s %s %s\n", x->res, x->a1, x->op, x->a2);
    }
}

```

optimizer.c

```

/*
 * Code optimizer for the quadruple table
 *
 * 1. Constant propagation and constant folding (data flow analysis, works across loops)
 * 2. Algebraic identities      x+0 x*1 x*0 x-x x^1 x^0 ...
 * 3. Strength reduction      x^2 -> x*x      x*2 -> x+x
 * 4. Merging of temporaries   t = a+b ; x = t -> x = a+b
 * 5. Common sub-expression elimination (inside a basic block)
 * 6. Copy propagation         t2 = t1
 * 7. Dead code elimination    unused temporaries, jumps to the next statement,
 *                             unreferenced labels, unreachable code
 *
 * The passes are repeated until nothing changes, because one pass often creates a
 * chance for another one. Variables of the source program are treated as live at the
 * end of the program, so only temporaries and unreachable code are ever deleted.
 * All arithmetic uses 32 bit int semantics, like C++ on a machine with 32 bit int.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include "compiler.h"

#define MAXV 400                /* variables and temporaries */

enum { S_PROP, S_FOLD, S_ALG, S_STR, S_CSE, S_COPY, S_MERGE, S_DEAD, S_UNREACH, S_COUNT };

static const char *title[S_COUNT] = {
    "Constant propagation", "Constant folding", "Algebraic identities",
    "Strength reduction", "Common sub-expression elimination", "Copy propagation",
    "Merging of temporaries", "Dead code elimination", "Unreachable code elimination"
};
static int count[S_COUNT];

typedef int32_t i32;

/* ----- arithmetic */

static i32 wrap(long long v)
{
    return (i32)(uint32_t)v;
}

static i32 power(i32 base, i32 e)        /* e <= 0 gives 1 */
{
    uint32_t r = 1, x = (uint32_t)base;
    uint32_t n = e > 0 ? (uint32_t)e : 0;
    while (n) {
        if (n & 1) r *= x;
        x *= x;
        n >>= 1;
    }
    return (i32)r;
}

static int evaluate(const char *op, i32 a, i32 b, i32 *r)
{
    switch (op[0]) {
        case '+': *r = wrap((long long)a + b); return 1;
        case '-': *r = wrap((long long)a - b); return 1;
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    case '*': *r = wrap((long long)a * b); return 1;
    case '/': if (b == 0 || (a == INT32_MIN && b == -1)) return 0;
               *r = a / b; return 1;
    case '%': if (b == 0 || (a == INT32_MIN && b == -1)) return 0;
               *r = a % b; return 1;
    case '^': *r = power(a, b); return 1;
    }
    return 0;
}

static void setCopy(Quad *x, const char *v)
{
    char tmp[NAMELEN];
    strcpy(tmp, v); /* v may point inside x */
    strcpy(x->op, "=");
    strcpy(x->a1, tmp);
    x->a2[0] = '\0';
}

static int hasA1(const Quad *x)
{
    return defines(x) || isRel(x->op) || !strcmp(x->op, "print");
}

static int hasA2(const Quad *x)
{
    return isBin(x->op) || isRel(x->op);
}

static int nextLive(int i)
{
    for (i++; i < nq; i++)
        if (!q[i].del) return i;
    return -1;
}

static int labelIndex(const char *name)
{
    int k;
    for (k = 0; k < nq; k++)
        if (!q[k].del && !strcmp(q[k].op, "label") && !strcmp(q[k].res, name)) return k;
    return -1;
}

/* ----- 1. constant propagation and folding */

typedef struct { int k; i32 v; } Val; /* k: 1 = constant v, 2 = not a constant */

static char vname[MAXV][NAMELEN];
static int nv = 0;
static Val in[MAXQ][MAXV]; /* value of every variable before quad i */
static char reached[MAXQ];
static int succ[MAXQ][2];

static int varIndex(const char *n)
{
    int k;
    for (k = 0; k < nv; k++)
        if (!strcmp(vname[k], n)) return k;
    if (nv >= MAXV) {
        printf("Error: too many variables\n");
        exit(1);
    }
    strcpy(vname[nv], n);
    return nv++;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

static Val getVal(const Val *st, const char *s)
{
    Val r = { 2, 0 };
    if (isNum(s)) { r.k = 1; r.v = wrap(atoll(s)); }
    else if (isVar(s) || isTemp(s)) r = st[varIndex(s)];
    return r;
}

static void transfer(Val *st, const Quad *x)
{
    Val r = { 2, 0 }, a, b;
    i32 v;
    if (!defines(x)) return;
    a = getVal(st, x->a1);
    if (!strcmp(x->op, "=")) {
        if (a.k == 1) r = a;
    } else if (!strcmp(x->op, "neg")) {
        if (a.k == 1) { r.k = 1; r.v = wrap(-(long long)a.v); }
    } else {
        b = getVal(st, x->a2);
        if (a.k == 1 && b.k == 1 && evaluate(x->op, a.v, b.v, &v)) { r.k = 1; r.v = v; }
    }
    st[varIndex(x->res)] = r;
}

static int meet(Val *dst, const Val *src) /* returns 1 if dst changed */
{
    int v, changed = 0;
    for (v = 0; v < nv; v++) {
        if (dst[v].k == 2) continue;
        if (src[v].k == 2 || dst[v].v != src[v].v) { dst[v].k = 2; changed = 1; }
    }
    return changed;
}

static void substitute(const Val *st, char *name, int *changes)
{
    char num[32];
    Val r;
    if (!isVar(name) && !isTemp(name)) return;
    r = st[varIndex(name)];
    if (r.k != 1) return;
    sprintf(num, "%d", (int)r.v);
    strcpy(name, num);
    count[S_PROP]++;
    (*changes)++;
}

static int constantPropagation(void)
{
    Val out[MAXV];
    int i, v, s, first, changed, changes = 0;

    for (i = 0; i < nq; i++) { /* make a slot for every name */
        if (q[i].del) continue;
        if (defines(&q[i])) varIndex(q[i].res);
        if (hasA1(&q[i]) && (isVar(q[i].a1) || isTemp(q[i].a1))) varIndex(q[i].a1);
        if (hasA2(&q[i]) && (isVar(q[i].a2) || isTemp(q[i].a2))) varIndex(q[i].a2);
    }

    /* control flow graph */
    for (i = 0; i < nq; i++) {
        reached[i] = 0;
        succ[i][0] = succ[i][1] = -1;
        if (q[i].del) continue;
        if (!strcmp(q[i].op, "goto")) {
            succ[i][0] = labelIndex(q[i].res);

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    } else if (isRel(q[i].op)) {
        succ[i][0] = nextLive(i);
        succ[i][1] = labelIndex(q[i].res);
    } else {
        succ[i][0] = nextLive(i);
    }
}

first = nextLive(-1);
if (first < 0) return 0;
reached[first] = 1;
for (v = 0; v < nv; v++) { in[first][v].k = 2; in[first][v].v = 0; }

do {
    changed = 0;
    for (i = 0; i < nq; i++) {
        if (q[i].del || !reached[i]) continue;
        memcpy(out, in[i], sizeof(Val) * nv);
        transfer(out, &q[i]);
        for (s = 0; s < 2; s++) {
            int t = succ[i][s];
            if (t < 0) continue;
            if (!reached[t]) {
                reached[t] = 1;
                memcpy(in[t], out, sizeof(Val) * nv);
                changed = 1;
            } else if (meet(in[t], out)) {
                changed = 1;
            }
        }
    }
} while (changed);

for (i = 0; i < nq; i++) {
    Quad *x = &q[i];
    if (x->del) continue;
    if (!reached[i]) { x->del = 1; count[S_UNREACH]++; changes++; continue; }

    if (hasA1(x)) substitute(in[i], x->a1, &changes);
    if (hasA2(x)) substitute(in[i], x->a2, &changes);

    if (isBin(x->op) && isNum(x->a1) && isNum(x->a2)) {
        i32 r;
        if (evaluate(x->op, wrap(atoll(x->a1)), wrap(atoll(x->a2)), &r)) {
            char num[32];
            sprintf(num, "%d", (int)r);
            setCopy(x, num);
            count[S_FOLD]++; changes++;
        }
    } else if (!strcmp(x->op, "neg") && isNum(x->a1)) {
        char num[32];
        sprintf(num, "%d", (int)wrap(-atoll(x->a1)));
        setCopy(x, num);
        count[S_FOLD]++; changes++;
    } else if (isRel(x->op) && isNum(x->a1) && isNum(x->a2)) {
        i32 a = wrap(atoll(x->a1)), b = wrap(atoll(x->a2));
        int t = !strcmp(x->op, "<") ? a < b : !strcmp(x->op, ">") ? a > b :
            !strcmp(x->op, "<=") ? a <= b : !strcmp(x->op, ">=") ? a >= b :
            !strcmp(x->op, "==") ? a == b : a != b;
        if (t) { strcpy(x->op, "goto"); x->a1[0] = x->a2[0] = '\0'; }
        else x->del = 1;
        count[S_FOLD]++; changes++;
    }
}
return changes;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
/* ----- 2. algebraic identities */
```

```
static int algebraicIdentities(void)
{
    int i, changes = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        char o;
        int done = 1;
        if (x->del || !isBin(x->op)) continue;
        o = x->op[0];
        if (o == '+') {
            if (!strcmp(x->a2, "0")) setCopy(x, x->a1);
            else if (!strcmp(x->a1, "0")) setCopy(x, x->a2);
            else done = 0;
        } else if (o == '-') {
            if (!strcmp(x->a2, "0")) setCopy(x, x->a1);
            else if (!strcmp(x->a1, x->a2)) setCopy(x, "0");
            else done = 0;
        } else if (o == '*') {
            if (!strcmp(x->a1, "0") || !strcmp(x->a2, "0")) setCopy(x, "0");
            else if (!strcmp(x->a2, "1")) setCopy(x, x->a1);
            else if (!strcmp(x->a1, "1")) setCopy(x, x->a2);
            else done = 0;
        } else if (o == '/') {
            if (!strcmp(x->a2, "1")) setCopy(x, x->a1);
            else done = 0;
        } else if (o == '%') {
            if (!strcmp(x->a2, "1")) setCopy(x, "0");
            else done = 0;
        } else if (o == '^') {
            if (!strcmp(x->a2, "1")) setCopy(x, x->a1);
            else if (!strcmp(x->a2, "0")) setCopy(x, "1");
            else done = 0;
        }
        if (done) { count[S_ALG]++; changes++; }
    }
    return changes;
}
```

```
/* ----- 3. strength reduction */
```

```
static int strengthReduction(void)
{
    int i, changes = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        if (x->del || !isBin(x->op)) continue;
        if (x->op[0] == '^' && !strcmp(x->a2, "2") && !isNum(x->a1)) {
            strcpy(x->op, "*"); /* x ^ 2 -> x * x */
            strcpy(x->a2, x->a1);
        } else if (x->op[0] == '*' && !strcmp(x->a2, "2") && !isNum(x->a1)) {
            strcpy(x->op, "+"); /* x * 2 -> x + x */
            strcpy(x->a2, x->a1);
        } else if (x->op[0] == '*' && !strcmp(x->a1, "2") && !isNum(x->a2)) {
            strcpy(x->op, "+"); /* 2 * x -> x + x */
            strcpy(x->a1, x->a2);
        } else continue;
        count[S_STR]++; changes++;
    }
    return changes;
}
```

```
/* ----- uses of a name */
```

```
static int uses(const char *n)
{

```


Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

int i, c = 0;
for (i = 0; i < nq; i++) {
    if (q[i].del) continue;
    if (hasA1(&q[i]) && !strcmp(q[i].a1, n)) c++;
    if (hasA2(&q[i]) && !strcmp(q[i].a2, n)) c++;
}
return c;
}

/* ----- 4. merging of temporaries */

static int mergeTemps(void)
{
    int i, j, changes = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        if (x->del || !defines(x) || !isTemp(x->res)) continue;
        j = nextLive(i);
        if (j < 0) continue;
        if (!strcmp(q[j].op, "=") && !strcmp(q[j].a1, x->res) && uses(x->res) == 1) {
            strcpy(x->res, q[j].res);          /* t = a op b ; x = t */
            q[j].del = 1;
            count[S_MERGE]++; changes++;
        }
    }
    return changes;
}

/* ----- 5. common sub-expression elimination */

typedef struct { char op[8], a1[NAMELEN], a2[NAMELEN], holder[NAMELEN]; } Avail;
static Avail av[MAXQ];
static int  nav;

static void kill(const char *n)          /* n is assigned: forget what depends on it */
{
    int k;
    for (k = 0; k < nav; ) {
        if (!strcmp(av[k].a1, n) || !strcmp(av[k].a2, n) || !strcmp(av[k].holder, n))
            av[k] = av[--nav];
        else
            k++;
    }
}

static int commonSubexpressions(void)
{
    int i, k, changes = 0;
    nav = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        char op[8], a1[NAMELEN], a2[NAMELEN];
        int found = -1;
        if (x->del) continue;
        if (!strcmp(x->op, "label")) { nav = 0; continue; }      /* new basic block */
        if (!defines(x)) continue;
        strcpy(op, x->op); strcpy(a1, x->a1); strcpy(a2, x->a2);

        if (isBin(op)) {
            int comm = (op[0] == '+' || op[0] == '*');
            for (k = 0; k < nav && found < 0; k++) {
                if (strcmp(av[k].op, op)) continue;
                if ((!strcmp(av[k].a1, a1) && !strcmp(av[k].a2, a2)) ||
                    (comm && !strcmp(av[k].a1, a2) && !strcmp(av[k].a2, a1)))
                    found = k;
            }
        }
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        if (found >= 0) {
            char holder[NAMELEN];
            strcpy(holder, av[found].holder);
            setCopy(x, holder);
            count[S_CSE]++; changes++;
        }
        kill(x->res);
        if (found < 0 && isBin(op) && strcmp(a1, x->res) && strcmp(a2, x->res)) {
            strcpy(av[nav].op, op); strcpy(av[nav].a1, a1);
            strcpy(av[nav].a2, a2); strcpy(av[nav].holder, x->res);
            nav++;
        }
    }
    return changes;
}

/* ----- 6. copy propagation ----- */

static int copyPropagation(void)
{
    int i, j, changes = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        if (x->del || strcmp(x->op, "=") || !isTemp(x->res) || !isTemp(x->a1)) continue;
        for (j = 0; j < nq; j++) {
            /* temporaries are assigned only once */
            if (q[j].del) continue;
            if (hasA1(&q[j]) && !strcmp(q[j].a1, x->res)) strcpy(q[j].a1, x->a1);
            if (hasA2(&q[j]) && !strcmp(q[j].a2, x->res)) strcpy(q[j].a2, x->a1);
        }
        x->del = 1;
        count[S_COPY]++; changes++;
    }
    return changes;
}

/* ----- 7. dead code elimination ----- */

static int labelReferenced(const char *name)
{
    int j;
    for (j = 0; j < nq; j++)
        if (!q[j].del && (!strcmp(q[j].op, "goto") || isRel(q[j].op)) &&
            !strcmp(q[j].res, name)) return 1;
    return 0;
}

static int deadCode(void)
{
    int i, changes = 0, changed = 1;
    while (changed) {
        changed = 0;
        for (i = 0; i < nq; i++) {
            Quad *x = &q[i];
            int n;
            if (x->del) continue;
            n = nextLive(i);
            if (defines(x) && isTemp(x->res) && uses(x->res) == 0)
                x->del = 1; /* result never used */
            else if (!strcmp(x->op, "=") && !strcmp(x->a1, x->res))
                x->del = 1; /* x = x */
            else if ((!strcmp(x->op, "goto") || isRel(x->op)) && n >= 0 &&
                !strcmp(q[n].op, "label") && !strcmp(q[n].res, x->res))
                x->del = 1; /* jump to next quad */
            else if (!strcmp(x->op, "label") && !labelReferenced(x->res))
                x->del = 1; /* nobody jumps here */
            else
                continue;
        }
        if (changes != 0)
            changed = 1;
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        count[S_DEAD]++; changes++; changed = 1;
    }
}
return changes;
}

/* ----- driver */

void optimize(void)
{
    int round, c;
    for (round = 0; round < 20; round++) {
        c = constantPropagation();
        c += algebraicIdentities();
        c += strengthReduction();
        c += mergeTemps();
        c += commonSubexpressions();
        c += copyPropagation();
        c += deadCode();
        if (!c) break;
    }
}

void printOptimizationReport(void)
{
    int k, any = 0;
    printf("Optimizations applied\n");
    for (k = 0; k < S_COUNT; k++)
        if (count[k]) { printf("    %-36s : %d\n", title[k], count[k]); any = 1; }
    if (!any) printf("    none\n");
}

```

codegen.c

```

/*
 * Code generator : optimized quadruples -> 8086 style assembly code
 *
 * Registers : every variable of the source program owns a register (R0, R1, ...).
 *              Temporaries share the registers that follow: a temporary gets a free
 *              register when it is defined and gives it back after its last use, and a
 *              result may reuse the register of an operand that dies in the same quad.
 * Constants : a constant operand is first moved to a scratch register (MOV R, #n).
 * Power      : x ^ n is expanded into repeated MUL (small constant n) or into a loop.
 * Output     : the assembly code is printed and also written to output.asm
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include "compiler.h"

#define MAXR 600

static FILE *out;

static char vname[MAXR][NAMELEN];          /* source variables -> R0 .. R(nvar-1) */
static int  nvar = 0;

typedef struct { char name[NAMELEN]; int reg; int last; } Temp;
static Temp tmp[MAXQ];                      /* temporaries: register and last use */
static int  ntmp = 0;

static char busy[MAXR];                     /* register in use (temporaries only) */
static int  maxReg = -1;
static int  scratch[8], nscratch = 0;
static int  powLabels = 0;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

/* ----- output */

static void ins(const char *fmt, ...)
{
    char buf[200];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(buf, sizeof buf, fmt, ap);
    va_end(ap);
    if (buf[strlen(buf) - 1] == ':') {
        printf("%s\n", buf);
        fprintf(out, "%s\n", buf);
    } else {
        printf("    %s\n", buf);
        fprintf(out, "    %s\n", buf);
    }
}

/* ----- registers */

static int varReg(const char *n)
{
    int k;
    for (k = 0; k < nvar; k++)
        if (!strcmp(vname[k], n)) return k;
    return -1;
}

static void addVar(const char *n)
{
    if (isVar(n) && varReg(n) < 0) strcpy(vname[nvar++], n);
}

static Temp *findTemp(const char *n)
{
    int k;
    for (k = 0; k < ntmp; k++)
        if (!strcmp(tmp[k].name, n)) return &tmp[k];
    strcpy(tmp[ntmp].name, n);
    tmp[ntmp].reg = -1;
    tmp[ntmp].last = -1;
    return &tmp[ntmp++];
}

static int regOf(const char *n)
{
    return isTemp(n) ? findTemp(n)->reg : varReg(n);
}

static int allocReg(void)
{
    int r;
    for (r = nvar; r < MAXR; r++)
        if (!busy[r]) {
            busy[r] = 1;
            if (r > maxReg) maxReg = r;
            return r;
        }
    printf("Error: out of registers\n");
    exit(1);
}

static int newScratch(void)
{
    int r = allocReg();
    scratch[nscratch++] = r;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    return r;
}

static void loadTo(int r, const char *a)
{
    if (isNum(a)) ins("MOV R%d, #s", r, a);
    else if (regOf(a) != r) ins("MOV R%d, R%d", r, regOf(a));
}

static int operandReg(const char *a)          /* register that holds operand a */
{
    int r;
    if (!isNum(a)) return regOf(a);
    r = newScratch();
    ins("MOV R%d, #s", r, a);
    return r;
}

/* Register for the result of quad i. A temporary that dies in this quad lends its
   register to the result. */
static int destReg(int i, const Quad *x)
{
    Temp *t;
    const char *ops[2];
    int k;
    if (!isTemp(x->res)) return varReg(x->res);
    t = findTemp(x->res);
    ops[0] = x->a1; ops[1] = x->a2;
    for (k = 0; k < 2; k++)
        if (isTemp(ops[k]) && findTemp(ops[k])->last == i) {
            t->reg = findTemp(ops[k])->reg;
            return t->reg;
        }
    t->reg = allocReg();
    return t->reg;
}

static void finish(int i, const Quad *x, int d)
{
    const char *ops[2];
    int k;
    ops[0] = x->a1; ops[1] = x->a2;
    for (k = 0; k < 2; k++)
        if (isTemp(ops[k]) && findTemp(ops[k])->last == i && findTemp(ops[k])->reg != d)
            busy[findTemp(ops[k])->reg] = 0;
    if (isTemp(x->res) && findTemp(x->res)->last < 0) busy[d] = 0;    /* never used */
    for (k = 0; k < nscratch; k++) busy[scratch[k]] = 0;
    nscratch = 0;
}

/* ----- instructions */

static const char *mnemonic(char op)
{
    switch (op) {
        case '+': return "ADD";
        case '-': return "SUB";
        case '*': return "MUL";
        case '/': return "DIV";
        default : return "MOD";
    }
}

static const char *jumpFor(const char *rel)
{
    if (!strcmp(rel, "<")) return "JL";
    if (!strcmp(rel, ">")) return "JG";
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (!strcmp(rel, "<=")) return "JLE";
    if (!strcmp(rel, ">=")) return "JGE";
    if (!strcmp(rel, "==")) return "JE";
    return "JNE";
}

/* d = a1 ^ a2 */
static void genPower(int d, const Quad *x)
{
    int base = newScratch(), acc = newScratch();
    int e = isNum(x->a2) ? atoi(x->a2) : -1;

    if (isNum(x->a2) && e >= 2 && e <= 16) {                /* repeated multiplication */
        int k;
        loadTo(base, x->a1);
        ins("MOV R%d, R%d", acc, base);
        for (k = 1; k < e; k++) ins("MUL R%d, R%d", acc, base);
    } else if (isNum(x->a2) && e <= 1) {                    /* x ^ 1 or x ^ 0 */
        if (e == 1) loadTo(acc, x->a1);
        else ins("MOV R%d, #1", acc);
    } else {                                              /* loop */
        int cnt = newScratch(), zero = newScratch(), one = newScratch();
        int n = ++powLabels;
        loadTo(base, x->a1);
        loadTo(cnt, x->a2);
        ins("MOV R%d, #1", acc);
        ins("MOV R%d, #0", zero);
        ins("MOV R%d, #1", one);
        ins("P%d:", n);
        ins("CMP R%d, R%d", cnt, zero);
        ins("JLE P%d_end", n);
        ins("MUL R%d, R%d", acc, base);
        ins("SUB R%d, R%d", cnt, one);
        ins("JMP P%d", n);
        ins("P%d_end:", n);
    }
    ins("MOV R%d, R%d", d, acc);
}

static void genBinary(int d, const Quad *x)
{
    const char *m = mnemonic(x->op[0]);
    int r1 = isNum(x->a1) ? -1 : regOf(x->a1);
    int r2 = isNum(x->a2) ? -1 : regOf(x->a2);
    int commutative = (x->op[0] == '+' || x->op[0] == '*');

    if (r2 == d && r1 != d) {                            /* the result register holds operand 2 */
        if (commutative) {
            ins("%s R%d, R%d", m, d, operandReg(x->a1));
        } else {
            int s = newScratch();
            loadTo(s, x->a1);
            ins("%s R%d, R%d", m, s, d);
            ins("MOV R%d, R%d", d, s);
        }
    } else {
        loadTo(d, x->a1);
        ins("%s R%d, R%d", m, d, operandReg(x->a2));
    }
}

void generate(FILE *asmFile)
{
    int i, k;
    out = asmFile;

    /* registers of the variables (in order of first appearance) and last use of temporaries */

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

for (i = 0; i < nq; i++) {
    const Quad *x = &q[i];
    if (x->del) continue;
    if (defines(x)) addVar(x->res);
    if (defines(x) || isRel(x->op) || !strcmp(x->op, "print")) addVar(x->a1);
    if (isBin(x->op) || isRel(x->op)) addVar(x->a2);
    if (isTemp(x->a1)) findTemp(x->a1)->last = i;
    if ((isBin(x->op) || isRel(x->op)) && isTemp(x->a2)) findTemp(x->a2)->last = i;
}

for (i = 0; i < nq; i++) {
    const Quad *x = &q[i];
    int d = -1;
    if (x->del) continue;

    if (!strcmp(x->op, "label")) {
        ins("%s:", x->res);
    } else if (!strcmp(x->op, "goto")) {
        ins("JMP %s", x->res);
    } else if (isRel(x->op)) {
        int r1 = operandReg(x->a1), r2 = operandReg(x->a2);
        ins("CMP R%d, R%d", r1, r2);
        ins("%s %s", jumpFor(x->op), x->res);
    } else if (!strcmp(x->op, "print")) {
        if (isStr(x->a1)) ins("OUT %s", x->a1);
        else if (isNum(x->a1)) ins("OUT #s", x->a1);
        else ins("OUT R%d", regOf(x->a1));
    } else if (!strcmp(x->op, "=")) {
        d = destReg(i, x);
        loadTo(d, x->a1);
    } else if (!strcmp(x->op, "neg")) {
        d = destReg(i, x);
        loadTo(d, x->a1);
        ins("NEG R%d", d);
    } else if (x->op[0] == '^') {
        d = destReg(i, x);
        genPower(d, x);
    } else {
        d = destReg(i, x);
        genBinary(d, x);
    }
    finish(i, x, d);
}

printf("\nRegister Allocation\n");
for (k = 0; k < nvar; k++) printf("    %-10s -> R%d\n", vname[k], k);
if (maxReg >= nvar) printf("    temporaries -> R%d .. R%d\n", nvar, maxReg);
else printf("    temporaries -> none needed\n");
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Input (input.cpp)

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int a = 10;
    int b = 20;
    int c = 2 * 3 + a * 1;
    int d = pow(b, 2);
    int sum = 0, i = 1;
    while (i <= 5) {
        sum = sum + i * 2;
        i = i + 1;
    }
    int x1 = sum + i;
    int x2 = sum + i;
    int e = sum * 2;
    if (a < b) {
        cout << "b is greater" << endl;
    } else {
        cout << "a is greater" << endl;
    }
    cout << c + d << endl;
    return 0;
}
```

Additional input (input2.cpp)

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int n = 6, f = 1, k = 2;
    int p = pow(k, n);
    int q = n % 4 + (n - n);
    while (n > 1) {
        f = f * n;
        n = n - 1;
    }
    int r = f / 3 + f * 2;
    int s = f / 3 + f * 2;
    int t = -f + p * 0 + q;
    // a loop with a branch inside
    int j = 0;
    while (j < 4) {
        if (j % 2 == 0)
            cout << j * j << endl;
        else
            cout << f - j << endl;
        j = j + 1;
    }
    cout << r + s << endl;
    cout << t << endl;
    cout << p << endl;
    return 0;
}
```

input2.cpp contains a loop whose values are not known at compile time, an if-else inside the loop, a remainder, a division and a repeated expression.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Commands Used

```
lex scanner.l
yacc -d parser.y
cc lex.yy.c y.tab.c quad.c optimizer.c codegen.c -lfl -o minicompiler
./minicompiler input.cpp
```

Output:

```
● kushaal@Kushaals-MacBook-Air Assignment 9 % flex scanner.l
● kushaal@Kushaals-MacBook-Air Assignment 9 % yacc -d parser.y
● kushaal@Kushaals-MacBook-Air Assignment 9 % gcc lex.yy.c y.tab.c quad.c optimizer.c codegen.c -o minicompiler
● kushaal@Kushaals-MacBook-Air Assignment 9 % ./minicompiler input.cpp
----- PHASE 1: LEXICAL ANALYSIS -----
#include <iostream> - preprocessor directive
#include <cmath> - preprocessor directive
using - keyword
namespace - keyword
std - identifier
; - special character
int - keyword
main - identifier
( - special character
) - special character
{ - special character
int - keyword
a - identifier
= - assignment operator
10 - integer constant
; - special character
int - keyword
b - identifier
= - assignment operator
20 - integer constant
; - special character
int - keyword
c - identifier
= - assignment operator
2 - integer constant
* - arithmetic operator
3 - integer constant
+ - arithmetic operator
a - identifier
* - arithmetic operator
1 - integer constant
; - special character
int - keyword
d - identifier
= - assignment operator
pow - function call
( - special character
b - identifier
, - special character
2 - integer constant
) - special character
; - special character
int - keyword
sum - identifier
= - assignment operator
0 - integer constant
```

Mini compiler output for input.cpp (part 1 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

, - special character
i - identifier
= - assignment operator
1 - integer constant
; - special character
while - keyword
( - special character
i - identifier
<= - relational operator
5 - integer constant
) - special character
{ - special character
sum - identifier
= - assignment operator
sum - identifier
+ - arithmetic operator
i - identifier
* - arithmetic operator
2 - integer constant
; - special character
i - identifier
= - assignment operator
i - identifier
+ - arithmetic operator
1 - integer constant
; - special character
} - special character
int - keyword
x1 - identifier
= - assignment operator
sum - identifier
+ - arithmetic operator
i - identifier
; - special character
int - keyword
x2 - identifier
= - assignment operator
sum - identifier
+ - arithmetic operator
i - identifier
; - special character
int - keyword
e - identifier
= - assignment operator
sum - identifier
* - arithmetic operator
2 - integer constant
; - special character
if - keyword
( - special character

```

Mini compiler output for input.cpp (part 2 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

a - identifier
< - relational operator
b - identifier
) - special character
{ - special character
cout - output stream (function call)
<< - stream insertion operator
"b is greater" - string constant
<< - stream insertion operator
endl - stream manipulator
; - special character
} - special character
else - keyword
{ - special character
cout - output stream (function call)
<< - stream insertion operator
"a is greater" - string constant
<< - stream insertion operator
endl - stream manipulator
; - special character
} - special character
cout - output stream (function call)
<< - stream insertion operator
c - identifier
+ - arithmetic operator
d - identifier
<< - stream insertion operator
endl - stream manipulator
; - special character
return - keyword
0 - integer constant
; - special character
} - special character

```

----- PHASE 2: SYNTAX ANALYSIS -----
 Syntactically correct

```

----- SYMBOL TABLE -----
Name      Type      Bytes      Address      Value
a          int         4           1000         10
b          int         4           1004         20
c          int         4           1008         -
d          int         4           1012         -
sum        int         4           1016         0
i          int         4           1020         1
x1         int         4           1024         -
x2         int         4           1028         -
e          int         4           1032         -

```

Mini compiler output for input.cpp (part 3 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

----- PHASE 3: INTERMEDIATE CODE (TAC) -----

```

a = 10
b = 20
t1 = 2 * 3
t2 = a * 1
t3 = t1 + t2
c = t3
t4 = b ^ 2
d = t4
sum = 0
i = 1
L1:
  if i > 5 goto L2
  t5 = i * 2
  t6 = sum + t5
  sum = t6
  t7 = i + 1
  i = t7
  goto L1
L2:
  t8 = sum + i
  x1 = t8
  t9 = sum + i
  x2 = t9
  t10 = sum * 2
  e = t10
  if a >= b goto L3
  print "b is greater"
  print "\n"
  goto L4
L3:
  print "a is greater"
  print "\n"
L4:
  t11 = c + d
  print t11
  print "\n"

```

Mini compiler output for input.cpp (part 4 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

----- PHASE 4: OPTIMIZED CODE -----

```
a = 10
b = 20
c = 16
d = 400
sum = 0
i = 1
```

L1:

```
if i > 5 goto L2
t5 = i + i
sum = sum + t5
i = i + 1
goto L1
```

L2:

```
x1 = sum + i
x2 = x1
e = sum + sum
print "b is greater"
print "\n"
print 416
print "\n"
```

Optimizations applied

```
Constant propagation      : 11
Constant folding          : 6
Strength reduction        : 2
Common sub-expression elimination : 1
Merging of temporaries    : 5
Dead code elimination     : 8
Unreachable code elimination : 2
```

Instructions: 36 -> 20

Mini compiler output for input.cpp (part 5 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

----- PHASE 5: TARGET CODE (8086 style) -----

```
MOV R0, #10
MOV R1, #20
MOV R2, #16
MOV R3, #400
MOV R4, #0
MOV R5, #1
```

L1:

```
MOV R9, #5
CMP R5, R9
JG L2
MOV R9, R5
ADD R9, R5
ADD R4, R9
MOV R9, #1
ADD R5, R9
JMP L1
```

L2:

```
MOV R6, R4
ADD R6, R5
MOV R7, R6
MOV R8, R4
ADD R8, R4
OUT "b is greater"
OUT "\n"
OUT #416
OUT "\n"
```

Register Allocation

```
a      -> R0
b      -> R1
c      -> R2
d      -> R3
sum    -> R4
i      -> R5
x1     -> R6
x2     -> R7
e      -> R8
temporaries -> R9 .. R9
```

Assembly code written to output.asm

Mini compiler output for input.cpp (6 of 6)

Phase 1 lists each token with its class. Phase 2 prints Syntactically correct along with the symbol table (type, size, address and initial value). Phase 3 shows the three address code and Phase 4 the optimized code. Phase 5 is the 8086-style assembly code, which is also saved in output.asm.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

In the optimized code, $c = 2*3 + a*1$ is folded to 16 and $d = \text{pow}(b, 2)$ to 400. Since $a < b$ is known to be true, the else part is removed and $c + d$ is printed as the constant 416. Inside the loop $i*2$ became $i+i$, and the second sum $+ i$ is replaced by $x2 = x1$ (common sub-expression). The number of instructions dropped from 36 to 20.

The accepted C++ subset consists of `#include` and `using namespace` lines, `int main()`, `int` declarations, assignments, `if / else`, `while`, `cout << ... << endl` (also `std::cout / std::endl`), the operators `+` `-` `*` `/` `%` and `pow(x, n)` for power, and the relational operators `<` `<=` `>` `>=` `==` `!=`. Declarations are block scoped.

Additional Test Cases:

```

● kushaal@Kushaals-MacBook-Air Assignment 9 % cat input3.cpp
#include <iostream>
using namespace std;

int main() {
    int a = 5;
    int b = a + 2
    cout << b << endl;
    return 0;
}
⊗ kushaal@Kushaals-MacBook-Air Assignment 9 % ./minicompiler input3.cpp
----- PHASE 1: LEXICAL ANALYSIS -----
#include <iostream> - preprocessor directive
using - keyword
namespace - keyword
std - identifier
; - special character
int - keyword
main - identifier
( - special character
) - special character
{ - special character
int - keyword
a - identifier
= - assignment operator
5 - integer constant
; - special character
int - keyword
b - identifier
= - assignment operator
a - identifier
+ - arithmetic operator
2 - integer constant
cout - output stream (function call)

----- PHASE 2: SYNTAX ANALYSIS -----
Syntactically incorrect: syntax error near 'cout' at line 7

```

Error handling: syntax, semantic and lexical errors

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Learning Outcomes:

- Understood how the compiler phases (lexical analysis, syntax analysis, intermediate code generation, optimization and code generation) fit together into one working compiler.
- Learnt to use Lex to recognize C++ tokens and pass them to the Yacc parser, and to report lexical errors such as unknown characters and unterminated strings.
- Learnt to write Yacc rules with precedence and associativity for expressions and for declaration, assignment, conditional and looping statements, and to build a symbol table with block scope and semantic checks.
- Learnt to generate three address code with temporaries and labels through syntax directed translation.
- Learnt to apply constant propagation and folding (with data flow analysis so that loops are handled correctly), algebraic identities, strength reduction, common sub-expression elimination and dead code elimination on three address code.
- Learnt to generate 8086-style assembly code with register allocation, comparison and jump instructions, and to build Lex and Yacc programs together with C code.